Eureka Technology

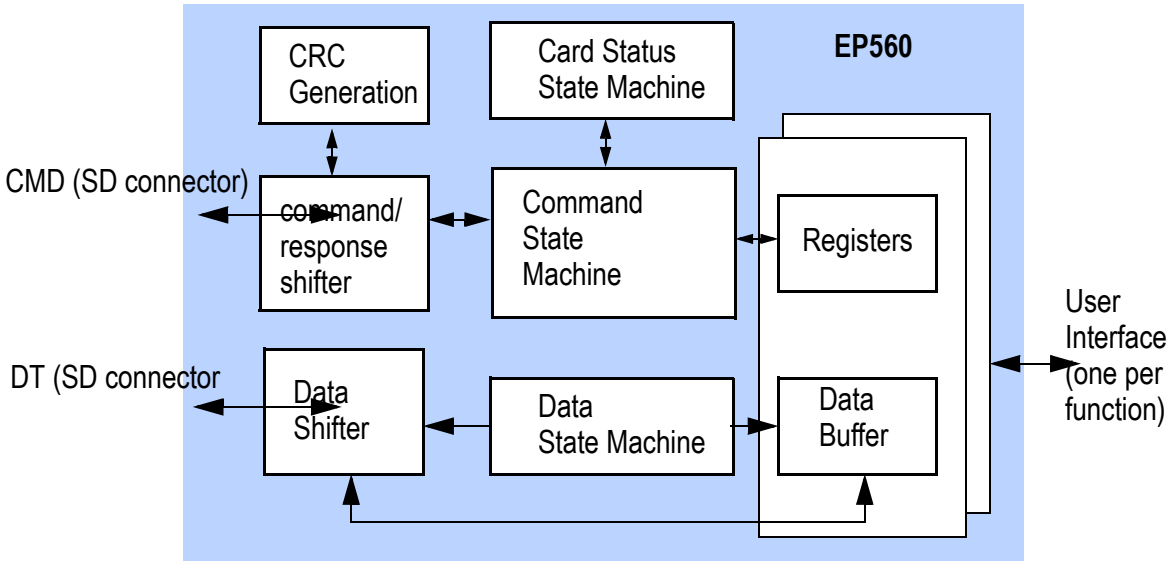
EP560 SDIO/SD memory Slave Controller

Data Sheet

FEATURES

- Compatible with SD/SDIO specification 2.0 with 1 and 4 bit data transfer.
- Provides SD interface to peripheral or memory device through a simple address/ data interface.
- Support SD, SPI and optional MMC bus protocol.
- Support for both standard capacity and high capacity (SDHC) memory cards.
- Supports high speed mode up to maximum transfer rate of 25Mbyte/sec.
- Simple 32-bit user interface for each function.
- Each function includes up to 4096 bytes of data buffer.
- Supports maximum block size up to 2048 bytes.
- Process most SD/SDIO commands automatically without user interference.
- Contains SD memory/SDIO standard slave register set.
- Hardware CRC generation and detection.
- Supports multi-function SD cards, suspend and resume, read wait, block transfers, and SDIO interrupts.
- Built in write protection features (permanent and temporary).
- Password Protection for SD cards.
- Static inputs allow user to define application specific register data.
- Option to support standard CPU buses such as AMBA AHB.
- Option to support MMC4.2 with 8-bit data width

BLOCK DIAGRAM



4962 El Camino Real, Los Altos, CA 94022, USA
http://www.eurekatech.com

DESCRIPTION

The SD Slave Controller is designed to reside within an SD memory, SDIO, or SD Combo Card. It serves as an interface between the SD bus and user logic that provides the actual function of the card. It is designed to integrate with user logic to make various devices using the SD bus protocol, such as storage or wireless network card.

The SD slave controller supports both 1 and 4 bit SD interface (up to 8 bits in optional MMC support) and SPI mode. Data rate of up to 25Mbyte/sec (200Mbs) can be realized with SD interface. Features such as plug and play, auto-detection, error correction, write protection are standard with SD card interface and are supported.

As a slave device, the SD slave controller receives commands from the host through the SD interface. Most of the commands are processed locally by the controller without any help from the user logic. The majority of the standard SD register set is also implemented within the slave controller and process by the core without help from the user logic.

In case of memory or IO access that needs to be forwarded to the user logic, the slave controller handles all the SD bus protocol and presents the request to the user logic as simple read and write request through parallel address and data buses. Burst transfer of up to 2048 bytes per transfer and user defined wait states are supported on the user interface to maximize data bandwidth. The slave controller also contains data buffer to match the speed differences between the user interface and the SD interface. It allows a much more efficient use of the user interface.

SD Combo card and multi-function cards are supports by the EP560. With Combo card and multi-function, one user interface is dedicated for each function so all functions can operate in parallel.

With the EP560, SD card design can be realized with very little development cost. The designer can add SD memory and SDIO interface capability to the design by simply adding the EP560 module without changing the rest of the system architecture.

1 Signal Descriptions

The following tables listed all the external I/O signals of SD slave controller. All signals with "#" or "_B" suffix are active low signals.

The following table describes the SD card interface signals between the controller and the SD connector

Table 1: SD Card Interface

Name	Type	Descriptions
SDCLK	I	Clock input from SD card interface.
CMD	I/O	Command and response on SD interface.
DT[3:0] or DT[7:0]	I/O	Data line. For SD memory and SDIO, The EP560 supports 1 bit and 4 bit data transfer. If the optional MMC feature is implemented, DT bit will be 8 bit wide to support 8-bit MMC transfer.

The following table describes the generic user interface signals with the controller function as a SD memory or MMC core. These signals are not present is the core is configured as SDIO only.

Table 2: User Interface Signals for SD memory/MMC

Signal	Type	Description
ABORT#	O	Asserted by the SD controller to abort the current transaction.
ADDR[40:0]	O	Byte address of the memory location to be accessed. 41 bits of address is provided to support the maximum size of MMC card. For any user logic that support less than the maximum size, the upper bits of the address can be ignored. For example, user logic with 1Gbyte of address space only need to connect to ADDR[29:0].

Table 2: User Interface Signals for SD memory/MMC

Signal	Type	Description
ADS#	O	Address strobe. Output to the user interface to start an access. The access type are provided in the CS# and WR signals. Other information such as address, byte enable and size information are valid from ADS# assertion to the last data transfer acknowledge.
BE#[3:0]	O	Byte enable. Indicates the byte(s) to be transferred.
BLAST#	O	Burst Last. This signal is asserted by the controller to the user interface to indicate the last data transfer of a burst sequence. This signal is valid one cycle after ADS# assertion.
CS#[1:0]	O	Chip select. This signal indicates the different address spaces accessed to the user interface. It has the same timing as the address output. Only one bit can be asserted for each access. Bit 0 is asserted to indicate SD memory or MMC access. Bit 1 is asserted to indicate special access.
DRD[31:0]	I	Read data. This bus provides the data read from the user interface to the controller core.
DWR[31:0]	O	Write data from the controller core to the user interface.
ERROR	I	Status input. This signal is used when RDY# is asserted to indicates error for erase or write transaction.

Table 2: User Interface Signals for SD memory/MMC

Signal	Type	Description
RDY#	I	Data ready or command acknowledge. This is the response signal from the user interface in response to ADS# assertion. For commands that involve data transfer, this signal indicates a valid data transfer on the user interface. For read access, valid read data is presented on DRD at the same time as RDY#. For write access, DWR is accepted by the core when RDY# is asserted. For burst data transfer, RDY# is asserted for one cycle for each data word. For erase and CSA access commands, the RDY# signal must be asserted with a certain clock limit. For other commands, user can assert RDY# at its own timing.
SIZE[9:0]	O	Transfer size. This signal indicates the number of 32-bit to be transferred for the ADS# command.
WR	O	Output to user interface to indicate read or write access. This signal is high for write and low for read.

The following table describes the generic user interface signals with the controller function as a SDIO code. One set of user interface signal is implemented for each function. The letter “n” in the signal name is replaced by the function number. These signals are not present if the core is configured not to support SDIO function. When SDIO function is supported, a minimum of function 0 and function 1 are present so the I0_xxx and I1_xxx signals are present.

Table 3: User Interface Signals for SDIO

Signal	Type	Description
In_ABORT#	O	Asserted by the SD controller to abort the current transaction.
In_ADDR[31:0]	O	IO address to the user logic

Table 3: User Interface Signals for SDIO

Signal	Type	Description
In_ADS#	O	Address strobe. Output to the user interface to start an access. The access type are provided in the CS# and WR signals. Other information such as address, byte enable and size information are valid from In_ADS# assertion to the last data transfer acknowledge.
In_BE#[3:0]	O	Byte enable. Indicates the byte(s) to be transferred.
In_BLAST#	O	Burst Last. This signal is asserted by the controller to the user interface to indicate the last data transfer of a burst sequence. This is signal is valid one cycle after In_ADS# assertion.
In_CS#[1:0]	O	Chip select. This signal indicates the different address spaces accessed to the user interface. It has the same timing as the address output. Only one bit can be asserted for each access. Bit 0 is asserted to SDIO space access Bit 1 is asserted to indicate CSA access. This occurs only to function 0.
In_DRD[31:0]	I	Read data. This bus provides the data read from the user interface to the controller core.
In_DWR[31:0]	O	Write data from the controller core to the user interface.
In_RDY#	I	Data ready or command acknowledge. This is the response signal from the user interface in response to ADS# assertion. For read access, valid read data is presented on In_DRD at the same time as In_RDY#. For write access, In_DWR is accepted by the core when In_RDY# is asserted. For burst data transfer, In_RDY# is asserted for one cycle for each data word.

Table 3: User Interface Signals for SDIO

Signal	Type	Description
In_SIZE[9:0]	O	Transfer size. This signal indicates the number of 32-bit to be transferred for the In_ADS# command.
In_WR	O	Output to user interface to indicate read or write access. This signal is high for write and low for read.

Table 4: System and Static Signals

Signal	Type	Description
CARD_ECC _DISABLED	I	Static input from the user logic to indicate if ECC is implemented. This signal is used for bit 14 of the CSR.
CARD_ECC _FAILED	I	Input from the user to indicate ECC failure. The core sets bit 21 of the CSR when this input is high.
CCCR[152:0]	I	Static input present only with SDIO function. Lower 153 bits of Card Common Control Registers read only values for SDIO
CCCR_OUT [153:17]	O	Present only with SDIO function. Output from CCCR register, reflect the values of the corresponding bits in the CCCR register. Only the following bits which are writable by the host have valid information. All other bits are hard-wired to “0”. The valid bits are: 153, 145, 143:128, 107:104, 97, 69, 63, 61, 57:56, 51:48, 39:32, 23:17.
CCCR_UPP ER[127:0]	I	Static input present only with SDIO function. Upper most 128 bits of Card Common Control Registers read only values for SDIO
CID[127:0]	I	Static input present with SD memory and MMC. Card Identification Register (CID) read only values, for SD memory.

Table 4: System and Static Signals

Signal	Type	Description
CSD[127:0]	I	Static input present with SD memory and MMC. Card Specific Data Register read only values, for SD Memory.
DSR[15:0]	O	Present with SD memory and MMC. Output of the Driver Stage Register. DSR is written by the host as SD memory command.
EPS1	O	Present only with SDIO function. Output from the FBR1 register reflecting the value of the EPS bit (bit 17)
EPS2 to EPS7	O	These signals are present only if SDIO function 2 to 7 are implemented. They are the value of the EPS bit of the corresponding FBR register.
FBR1[143:0]	I	Static input present only with SDIO function. Function Basic Register read only values for SDIO Function 1.
FBR2 to FBR7	I	These signals are present only if SDIO function 2 to 7 are implemented. They are the read only values of the FBR for function 2 to 7.
HRESET#	I	This signal is present in all modes of operation. Hardware reset. Indicates when power is stable and the controller can start operation.
OCR_SDIO[23:0]	I	Static input present only with SDIO function. Operation Condition Register read only values, for SDIO.
OCR_MEM[31:0]	I	Static input present with SD memory and MMC. Operation Conditions Register read only values, for SD memory.

Table 4: System and Static Signals

Signal	Type	Description
OE_CARD_DETECT	O	Present with SD memory and SDIO. This output is controlled by the set_clr_card_detect SD commands. This signal should be used at the chip to enable(1) or disable(0) a tri-state buffer. The tri-state buffer output (drives high if it is enabled) should be connected to a weak pull up resister connected to DT3.
PWDIN[127:0]	I	Present with SD memory and MMC. Current password stored by the card. Password should be stored in non-violatile memory within the card.
PWD_LEN[7:)]	I	Present with SD memory and MMC. Length of the current password. Password length should be stored in non-violatile memory within the card.
SCR[63:0]	I	Statc input present with SD memory. SD Configuration Register read only values, for SD memory.
SD_DEV[1:0]	I	Static input present only when the core supports both SD memory and SDIO. Indicates if SD memory or SDIO function is enabled. Valid combinations are: 01: SD memory is enabled. 10: SDIO is enabled. 11: Both SD memory and SDIO are enabled.
SDIO_CRD_RDY	I	Present only with SDIO function. SDIO card initialization. This signal is high to indication initialization complete and the card is ready.
SRESETI#	O	Present only with SDIO function. SDIO soft reset output
SRESETM#	O	Present with SD memory and MMC. SD memory soft reset output

Table 4: System and Static Signals

Signal	Type	Description
THICK_CARD	I	Static input present only with SD memory. to indicate if the card is a thick (2.1mm) card. It should be “1” for thick card and “0” for thin card.

1.1 Optional MMC Support Signals

The following signals are present if the optional MMC feature of the EP560 is implemented. This optional feature is included in the core only if this feature is requested when ordering the IP core.

Table 5: Signals for MMC Support

Name	Type	Descriptions
DT[7:0]	I/O	Data line. When MMC support is added, the data line changes from 4 to 8 bit to support 8 bit data transfer.
ECSD_A[7:0], ECSD_B[31:0], ECSD_C[7:0] --- ECSD_Q[7:0]	I	Extended CSD read only data, static input. Extended CSD is a register specific to MMC mode. The register contains mostly read only data and the read only values are provided to the core through this static input. User can hardware these values according to the characteristics of the card.
ECSDR	O	POWER_CLASS from Extended CSD
ECSDS	O	HS_TIMING from Extended CSD

Table 5: Signals for MMC Support

Name	Type	Descriptions
RD_THD[2:0]	I	Read Threshold for stream data read, static input. In steam read mode, in order to prevent data underflow, the user may want to buffer enough data inside the EP560 before the first data is returned to the SD card interface. The encoding of the RD_THD is the following: 000: Buffer 16 words before sending data to host. 001: Buffer 32 words before sending data to host. 010: Buffer 64 words before sending data to host. 011: Buffer 128 words before sending data to host. 111: No Buffer, send data to host as soon as first word arrive from user interface. Others: Reserved
SEL_MMC	I	Select MMC, static input. This signal is present if the core supports both SD mode and MMC mode. This signal is not needed if the core supports only SD or MMC but not both. This signal is “1” to indicate MMC mode and “0” to indicate SD mode. This signal must remain unchanged after reset.

1.2 Optional AHB Interface

If the optional AHB interface is used, the user interface will be replaced by the following table. Multiple AHB master interface is implemented if the core is configured to support multiple functions or as combo card. This optional feature is included in the core only if this feature is requested when ordering the IP core.

Table 6: AHB Bus Master Interface

Name	Type	Descriptions
HADDR[31:0]	O	AHB bus address driven by the master

Table 6: AHB Bus Master Interface

Name	Type	Descriptions
HBURST[2:0]	O	Burst length. Each data burst may be 4, 8 or 16 data beat long, depending on burst internal buffer size and actual number of data to be transferred. Both cacheline wrap and linear increment are allowed on the AHB bus but the AHB master generates liner increment transfer only.
HBUSREQ	O	Bus request. Indicates the master's intention of acquiring the AHB bus to transfer data.
HGRANT	I	Bus grant. Driven by the system arbiter to indicate to the master that it has bus ownership.
HLOCK	O	Bus lock. The master does not request lock access so this signal is driven low by the master.
HROT[3:0]	O	AHB bus protection mode. It is hardwired to "0011".
HRDATA[31:0]	I	Read data
HREADY	I	Transfer ready. The signal is driven by the slave to signal data transfer.
HRESP[1:0]	I	Bus response. Four different responses are allowed: OKAY, ERROR, RETRY and SPLIT.
HSEL[1:0]	O	Chip select. This signal indicates the different address spaces accessed to the user interface. Only one bit can be asserted for each access. Meaning of each HSEL is the same as the CS# or In_CS# signals of the respective port.
HSIZE[2:0]	O	Data bus size. It indicates the data size is 8-bit, 16-bit or 32-bit.
HTRANS[1:0]	O	Transfer type. It can be NONSEQUENTIAL, SEQUENTIAL, IDLE or BUSY.
HWDATA[31:0]	O	Write data.
HWRITE	O	Read/write indicator. It is high for write and low for read.

2 User Interface

This section describes the operation of the user interface. The user interface operates at the domain of the SD Clock (SDCLK). All inputs are sampled with the rising edge of SDCLK and all output signals are driven from the rising edge of SDCLK.

Most of the interaction between the SD bus and the controller is handled by the SD slave controller automatically without any action required by the user logic. Only a few transactions such as memory read, write or erase are forwarded to the user logic through the user interface.

The following sections describes all of those interactions between the SD slave controller and the user logic. SD bus protocol or commands not described in this section are handled internally by the slave controller thus not a concern of the user interface.

2.1 User Commands and Basic Protocol

SD Slave and User interaction is initiated by the SD Slave asserting ADS#. The commands can be a data transfer request or other SD commands requiring response from the user logic. The RDY# is asserted by the user logic when data is ready for data transfer. The RDY# assertion timing depends on the type of command. For timing and details refer to the sections below.

2.2 Single Read

All read accesses are originated from the SD interface by the system host. The slave controller receives the command from the host and generate the proper responses to the host and performs CRC checking automatically. If it is a read access to user area, the controller forwards the read request to the user logic on a block by block basis. Read data returned from the user logic are first stored in the controller and then transferred to the SD interface with CRC generated automatically by the controller. All SD bus protocol, including start bit, end bit, etc., are handled by the controller.

A read operation to the user interface is initiated by the SD slave by asserting ADS#. CS# signal is "10" to indicate normal access and “01” to indicate special access. The WR signal is low to indicate a read access. The address to access is on the ADDR signal. SIZE is 001 (hex) to indicate one word is being accessed. All the fore mentioned signals are valid at the same time that ADS# is asserted and remain valid until the end of the transfer.

After ADS# is asserted, user may proceed with the read operation and returns the read data to the slave controller when it is ready. RDY# is asserted by the user

logic to signal read data is available on the DRD signal. The number of clock cycles between ADS# assertion and RDY# assertion is controlled by the user but must be within the time limit specified below.

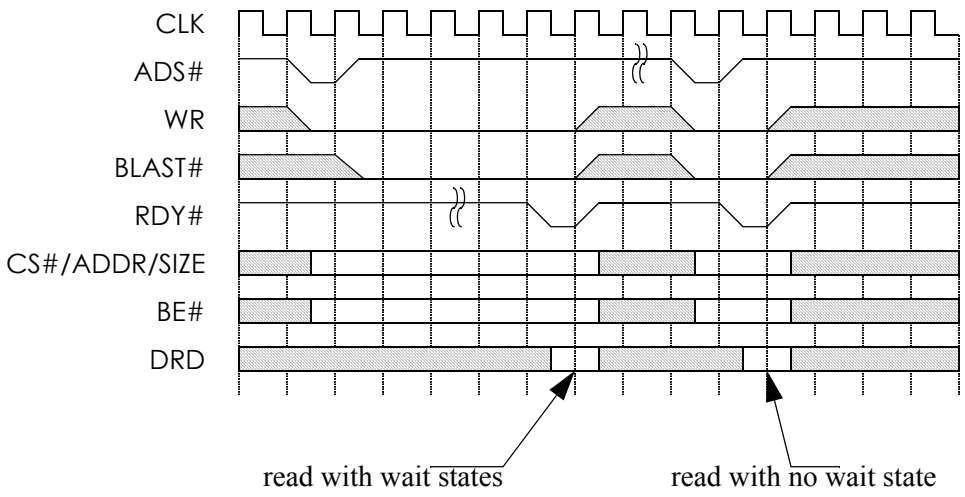


Figure 2.1 Single data read access

2.3 Burst Read

A burst read access is very similar to that of a single read. The read access is initiated in the same exact way as the single read access, except that SIZE is not 001 (hex) but the number of words to be transferred and BLAST# is asserted at the last data phase of the burst read. RDY# is asserted by the user logic when each read data is available on the DRD signal. User may assert RDY# continuously or insert wait state in between subsequent data if needed by de-asserting RDY# in between data transfers. The transfer ends when the number of data being transferred equals the number specified in SIZE. The controller also asserted BLAST# to signal the

last data phase.

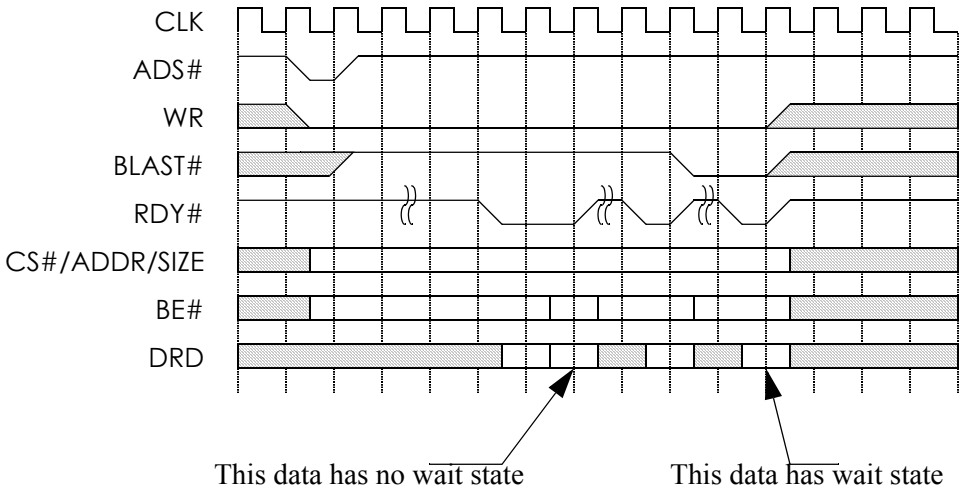


Figure 2.2 Burst data read access

2.4 Read Access Time Limit

For single or burst data read access to standard capacity SD Memory card, RDY# of the last data must be asserted within a certain time limit from ADS# assertion. The time limit is

((100 * ((TAAC * fpp) + (100* NSAC))) - 1) cycles or 100 ms, whichever is less.

The TAAC and NSAC values are defined in the CSD register in later section. The value of TAAC can range from 1ns to 80ms. NSAC can range from 0 to 255. fpp is the frequency of the SD clock.

For the read access of high capacity SD Memory cards RDY# of the last data must be asserted within 100 ms. from ADS# assertion.

For SDIO, single read transfer must finish within 60 cycles of In_ADS# assertion. For SDIO burst read transfer, the transfer of all data must be completed within 1 second.

2.5 Single Write

All write accesses are originated from the SD interface by the system host. The slave controller receives the command from the host and generate the proper responses to the host and performs CRC checking automatically. If it is a write access to user area, the controller first post each block of write data into its write buffer, checks CRC and then forwards the write request to the user logic on a block by block basis. The controller can buffer up to 4096 bytes of data and capable of writing data to the user logic at the same time receiving data from the host. All SD bus protocol, including status bits, busy, etc., are handled by the controller.

A single write operation is similar to the single read operation. The involved signals are the same at the time ADS# is asserted by the SD Slave, with the exception of WR, which is high to indicate a write operation, and data is valid on DWR signal. RDY# is asserted by the user logic when DWR is accepted for write. Similar to single read, the transfer ends when RDY# is asserted. ERROR signal must be inactive during the entire transfer to indicate no write protection error. The number of clock cycle between ADS# assertion and RDY# assertion is controlled by the user.

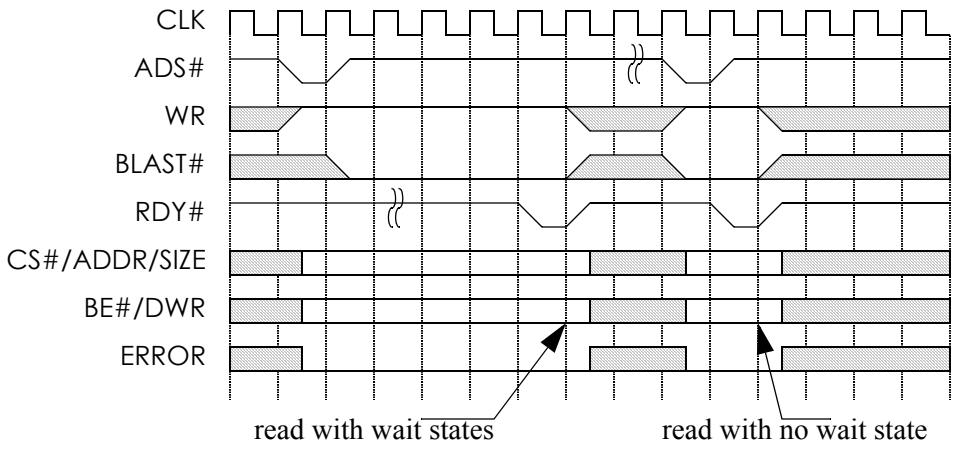


Figure 2.3 Single data write access

2.6 Burst Write

A burst write operation is similar to burst read as single write is to single read. The

involved signals sent at the time ADS# is asserted by the SD Slave as in the case of single write, except BLAST# and SIZE. SIZE indicates the number of 32-bit words to be transferred and BLAST# will be asserted when the last 32-bit word is sent on DWR. SIZE is valid from the time ADS# is asserted until the end of the transfer. RDY# is asserted by the user logic when the data is accepted. The controller drives the next write data on the DWR in the cycle following RDY# assertion. RDY# may be asserted continuously by the user if no wait state is needed or insert wait states in between RDY# assertion. BLAST# is asserted by the SD Slave when the last word is sent on DWR and the transfer ends when both BLAST# and RDY# are asserted at the same cycle.

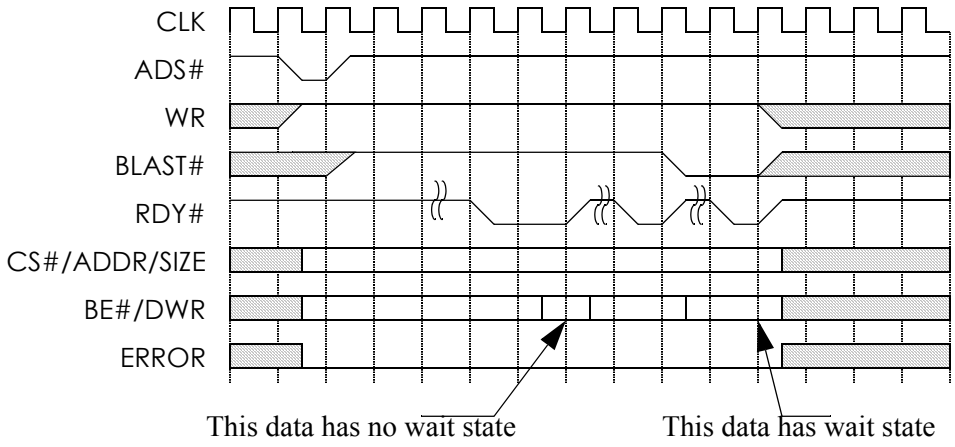


Figure 2.4 Burst data write access

2.7 Write Protection Error

The user logic can signal write protection error on a write transaction. ERROR is asserted by the user when RDY# is asserted in response to memory write to indicate write protection error. When write protection error occurs, it triggers the following events:

1. The bus transfer on the user interface terminates immediately. No more data will be transfer even if it is a burst write.
2. The core sets the WP_VIOLATION_ERROR bit (bit 26) of the Card Status Register (CSR).

3. If it is a multiple block write, remaining data blocks already received by the core will not be written to the user interface and buffered data will be discarded.
4. If the SD interface continue to shift in write data, the core will respond with no ECC status (status="111" with no start bit) to indicate write error.

The following diagram shows the user interface on write protection violation.

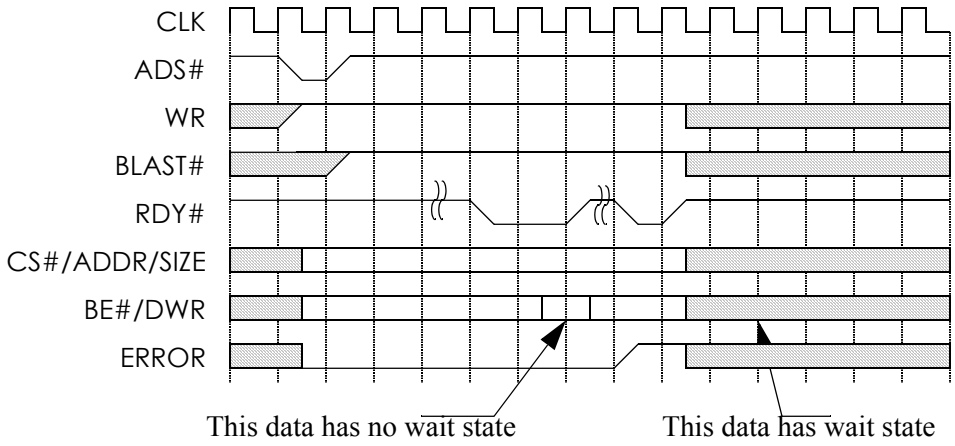


Figure 2.5 Write protection violation

2.8 Write Access Time Limit

For write access of a standard capacity SD Memory cards, RDY# of the last data must be asserted within a certain time limit from ADS# assertion. The time limit is the lessor of the following two limits:

1. the read access time limit multiplied by R2W_FACTOR, or
2. 250ms

The read access time limit is defined in previous section. R2W_FACTOR is defined in CSD register and has a range from 1 to 32.

For high capacity SD Memory cards, RDY# shall be asserted within 250 ms.

For SDIO, single write transfer must finish within 60 clock cycles from ADS# assertion. After the write operation, a read may immediately follow if it is a read after write operation. IN this case, RDY# for the read must be asserted within 60

clock cycles of the ADS# assertion of the write transfer. For SDIO burst write transfer, the transfer of all data must be completed within 1 second.

2.9 Programming Command

This section applies to SD memory only and is not applicable to SDIO.

After all the data blocks associated with one SD write command has been sent out, the core issues a programming command to the user interface. Programming command is indicated by a single write command with CS#="01" and address equal to "08008030"(hex). User can terminate this write command with RDY# as in normal write cases. No error signaling is allowed.

During the time the core is waiting for RDY# assertion for the programming command, the core holds DT[0] lines to zero to indicate to the host that programming is in progress.

Programming command automatically generated by the core for SD memory write. It is not generated for CSA space write.

2.10 Request Abort

Any request can be aborted by the SD slave controller after the assertion of ADS#. There are two reasons an abort is issued by the SD slave controller. The first is for read transaction when the SD slave controller detects that read data is no longer needed by the host. The read abort is signaled by the assertion of ABORT# signal. The read terminates immediately at the assertion of ABORT#.

The second case is applicable only to SDIO. If in the middle of an SDIO transaction, the host issues a direct IO read/write command, the current transaction is aborted. The direct IO command has a timing requirement such that it cannot wait for the current transaction to finish and must be processed immediately. Therefore, the SD slave controller issues an abort by asserting In_ABORT#. The next command following is either a single read or write access. In_RDY# must be asserted within 60 cycles after In_ADS# assertion. In the case of a single write access, it may be followed by a read access to the same address to preform a read after write. In the read after write case, the In_RDY# for the read access (second access) must be asserted within 60 cycles of the In_ADS# of the write access (first access). The SD slave controller then resumes the previous transaction by asserting In_ADS# with the other signals to resume from where it left off at the time of abort.

The RDY# signal is ignored at the cycle when ABORT# is asserted. When an aborted transaction is resumed at the user interface, the slave controller resumes the transaction as if RDY# was not asserted when ABORT# was previously

asserted. The following diagram is an example of an abort case.

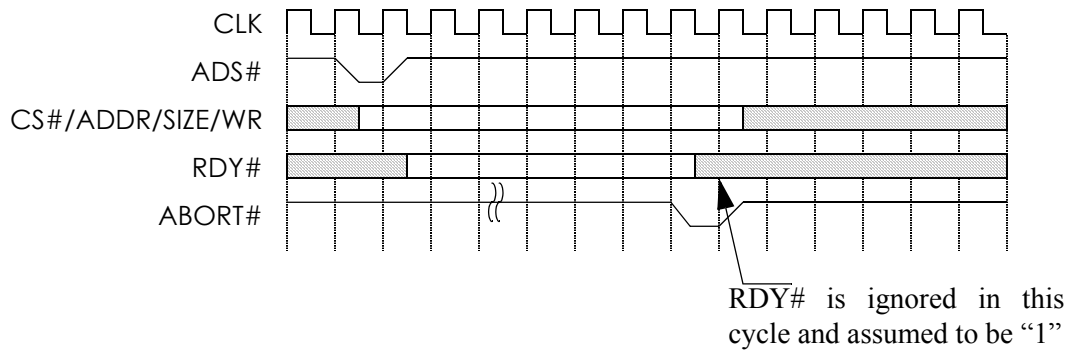


Figure 2.6 Transaction Abort

2.11 Access Address for SD memory

The access address contains additional information according to the CS# signal.

Table 7: SD memory Access Address

CS#[1:0]	Address	Note
10	All 32-bit are specified according to the SD command	Normal SD memory read or write

Table 7: SD memory Access Address

CS#[1:0]	Address	Note
01	Bit 31:7 = all zero Bit6:0 = register address	Fast IO access in MMC mode. Not active for SD memory.
	Bit 31:16 = 0800(hex) Bit 15:0 = 0000(hex)	SD register read
	Bit 31:16 = 0800(hex) BIt 15:0 = 8000(hex)	Erase start address, write only
	Bit 31:16 = 0800(hex) Bit 15:0 = 8010(hex)	Erase end address, write only
	Bit 31:16 = 0800(hex) Bit 15:0 = 8020(hex)	Erase execution, write only
	Bit 31:16 = 0800(hex) Bit 15:0 = 8030(hex)	Programming command, write only
	Bit 31:16 = 0800(hex) Bit 15:0 = 8040(hex)	Set Write Protect
	Bit 31:16 = 0800(hex) Bit 15:0 = 8050(hex)	Clear Write Protect
	Bit 31:16 = 0800(hex) Bit 15:0 = 8060(hex)	Argument for read Write Protect
	Bit 31:16 = 0800(hex) Bit 15:0 = 8070(hex)	Read Write Protect
	Bit 31:16 = 0800(hex) Bit 15:0 = 8080(hex)	Pre-write erase block count
	Bit 31:16 = 0800(hex) Bit 15:0 = 8090(hex)	CSD write
	Bit 31:16 = 0800(hex) Bit 15:0 = 8100(hex)	Force erase

Table 7: SD memory Access Address

CS#[1:0]	Address	Note
01	Bit 31:16 = 0800(hex) Bit 15:0 = 8110(hex)	Password Length
	Bit 31:16 = 0800(hex) Bit 15:0 = 8200(hex)	Card Password
	Bit 31:16 = 0800(hex) Bit 15:0 = 8300(hex)	Switch function argument
	Bit 31:16 = 0800(hex) Bit 15:0 = 8400(hex)	Switch function status block
	Bit 31:16 = 0810(hex) Bit 15:0 = 0000(hex)	CID programming. This access is possible only if the optional MMC feature is included.
	Bit 31:16 = 0810(hex) Bit 15:0 = 1000(hex)	MMC Interrupt. This access is possible only if the optional MMC feature is included.
	Bit 31 = 1	Read/write special block (CMD56). Argument bit 31:1 is outputted on bit 30:0 or address.

2.12 Access Address for SDIO

The access address contains additional information according to the CS# signal.

Table 8: Access Address for Function 0

CS#[1:0]	Address	Note
10	Bit 31:17 = all zero Bit 16:0 = CIS address	IO space access (CIS access for function 0)
01	Bit 31:24 = all zero Bit 23:0 = CSA index	CSA space read/write

The access address contains additional information according to the CS# signal.

Table 9: Access Address for Function 1 - 7

CS#[1:0]	Address	Note
10	Bit 31:17 = all zero Bit 16:0 = IO address	IO space read or write address
01	Reserved	

2.13 Pre-erased Setting prior to a Multiple Block Write Operation

This section applies to SD memory only and is not applicable to SDIO.

Before the SD Slave issues a write request, it may send a number of blocks to be erased before write operation. This pre-erase number of blocks is only valid for SD Memory writes. To send the command, ADS# is asserted by the SD Slave with CS#=01, Address = 08008080(hex) and WR="1". The number of blocks to erase is outputted on the DWR signal. The user logic asserts RDY# when it has received the value.

The address to start the pre-erase is not available until the write command is executed after the number of blocks to pre-erase has been sent. If the following command is not an SD Memory write, the user logic sets the number of pre-erase blocks to the default value of 1. If the number of write blocks transferred is less than the number of blocks specified to pre-erase, then the blocks erased and not written to are considered undefined. If more write blocks are sent than defined by the pre-erase number block, the block is erased as the new block is received.

2.14 Erase Operation

This section applies to SD memory only and is not applicable to SDIO.

An erase operation is executed through a sequence of three commands by the SD Slave: erase write block start, erase write block end, and erase execute. On the user interface, the erase commands are write command to specific address locations. The core first write the erase start address and then the erase end address. It is then followed by writing the erase execution command on the user interface.

The sequence is started by a write command on the user interface. ADS# is asserted by the SD Slave with CS#=01, Address = 08008000(hex) and WR=”1”. The starting address to be erased is outputted on the DWR signal. The user logic must verify that the erase address provided on DWR is valid. When RDY# is asserted by the user logic, ERROR is asserted by the user if the address is invalid.

ERROR is de-asserted if the address is valid. The RDY# signal must be asserted within 50 clock cycles from the time ADS# is asserted.

The second command in the sequence, erase write block end, is handled very similarly to the first. ADS# is asserted by the SD Slave with CS#=01, Address = 08008010(hex) and WR="1". The starting address to be erased is outputted on the DWR signal. Again, the user logic must verify that the address provided is valid. When RDY# is asserted by the user logic, ERROR is asserted by the user if the address is invalid. ERROR is de-asserted if the address is valid. The RDY# signal must be asserted within 50 clock cycles from the time ADS# is asserted.

The final command is erase execution. It is handled also as a write command at the user interface. ADS# is asserted by the SD Slave with CS#=01, Address = 08008020(hex) and WR="1". At this time the user logic begins erasing the blocks in the address range as provided on the previous erase start and erase end commands. The RDY# signal is asserted by the user logic when erase has completed. If blocks were skipped during erase due to the fact they were write protected, ERROR is asserted when RDY# is asserted by the user logic. If no blocks were skipped due in erase execution, ERROR is de-asserted.

The SD bus specification requires that the RDY# for erase execution must be asserted within 250ms from ADS# assertion.

The 3 command erase sequence is issued by the host to the slave controller. Receiving the erase address start or erase address end commands does not guarantee the erase execute command will be issued. If any of the first two commands results in error, the host may re-start the sequence again without finishing the previous sequence. User logic should erase its memory only at the receipt of the erase execute command.

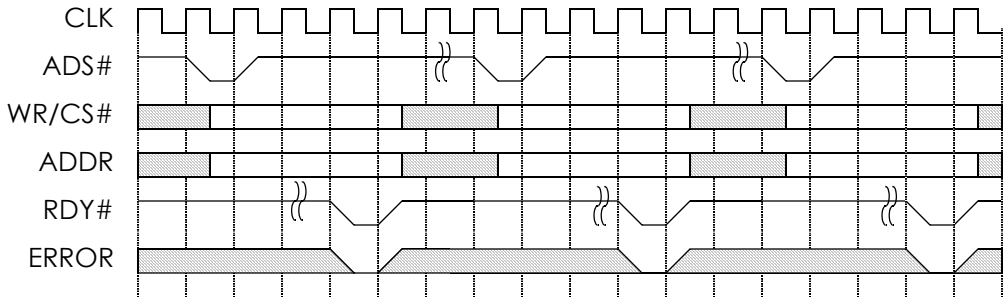


Figure 2.7 Erase Sequence

2.15 General Read Write (CMD56)

This section applies to SD memory only and is not applicable to SDIO.

In addition to the addressed read and write commands of SD Memory, there is also a special single block read/write command, listed as CMD56 in the SD Specification. The use of this block of data is user specific. With the exception of the 0th bit of the argument which indicates read or write, the rest of the bits are to be defined by the user. Therefore, the blocks of data are transferred as described below.

ADS# is asserted by the SD Slave with CS#=01. WR="1" for write and WR="0" for a read. Bit 31 of the address is "1" to indicate that this is the General Read Write (GEN_CMD) command. Bits 30 to 0 of the address signal correspond to Bits 31 to 1 of the argument of the command sent by the host. User logic must respond with RDY# assertion as in regular read or write cases.

2.16 Write Protect of Groups of sectors

This section applies to SD memory only and is not applicable to SDIO.

This feature is available only in standard capacity card

There are three types of write protection commands that the SD Slave can issue to the user logic, where sectors of the data can be write protected. The three commands are set write protection, clear write protection, and send write protection. The unit of a sector of data is called a write-protection group. The size of a write-protection group is set in the CSD.

The set write protection command sets the write protection of an addressed write-protect group. ADS# is asserted with CS#="01", address = 08008040 (hex), and WR="1". The address of the write-protect group is outputted on the DWR signal. The user logic asserts RDY# when it has finished setting the write protection.

The clear write protection command clears the write protection of an addressed write-protect group. ADS# is asserted with CS#="01", address = 08008050 (hex), and WR="1". The address of the write-protect group is outputted on the DWR signal. The user logic asserts RDY# when it has finished setting the write protection.

The send write protection command is a combination of two operations. These two operations are always executed together, with the SD Slave sending the write-protect address during the first operation and the user logic sending the write-protection bits during the second operation.

During the first operation the SD Slave sends a write-protect group address. ADS# is asserted with CS#="01", address = 08008060 (hex) and WR="1". This write-protect group address is outputted on the DWR signal. The user logic asserts

RDY# when it has received the value.

During the second operation, the SD Slave sends a request to obtain write-protection information. ADS# is asserted with CS#="01", address = 08008070 (hex) and WR="0". The user logic asserts RDY# when the write-protection information is on DRD. This 32-bit block contains 32 write protection bits, which represent 32 write protect groups starting at the address that was sent during the first operation).

2.17 Setting Password and Password Length

This section applies to SD memory only and is not applicable to SDIO.

The user logic stores and provides the current card password and the password length through the SD slave controller inputs PWDIN and PWD_LEN, respectively. These values must be stored in non-volatile storage, because the password and password length need to be stored after the card is powered off. A card is considered to have no password if the password length is 0. These values can be changed when the SD slave controller issues the request to do so.

The SD slave controller issues a set password request by asserting ADS#, CS#="01", address = 08008200 (hex) and WR="1". On the SIZE signal is the number of words in the transfer. This transfer may be issued as a single write transfer or burst write transfer depending on the size of the new password.

The SD slave controller issues a set password length request by asserting ADS#, CS#="01", address = 080081100 (hex) and WR="1". The length is always one word and is always transferred using single write transfer.

2.18 Force Erase

This section applies to SD memory only and is not applicable to SDIO.

When the SD slave controller issues a force erase command, all the contents in memory must be cleared, the password must be cleared (by setting the length to 0), and any write protection set using the method described in section 2.15 must be cleared as well.

The SD slave controller issues a force erase request by asserting ADS#, CS#="01", address = 08008100 (hex) and WR="1". The user logic asserts RDY# when it has completed the force erase as described above.

2.19 Switch Function

This section applies to SD memory only and is not applicable to SDIO.

The user logic is responsible for implementing the 512 bit SD Memory function

status block, which is ultimately sent back to the host when the host issues the switch function command. The structure of this data block can be found on pages 39-42 of the official SD Specification, Part 1, Rev 2.0. In order the handle this command, the user logic is sent a sequence of two requests by the SD slave controller.

The first request is a write request. ADS# is asserted with CS#="01", address = 08008300 (hex) and WR="1". On the DWR signal is the argument from the switch function command that was sent back to the host. Details on the argument can be found on page 64, table 4-28 of the official SD Specification, part 1 v2.0. When the user logic has written the data, RDY# is asserted to indicate the end of the first request.

The second request is a burst read request. ADS# is asserted with CS#="01", address = 08008400 (hex) and WR="0". SIZE is 010 (hex) to indicate 16 words of data for the transfer. The status block described above is sent at this time. When BLAST# and RDY# are asserted on the same cycle, the second request ends.

2.20 SD Status Register Access

This section applies to SD memory only and is not applicable to SDIO.

All the SD registers are contained internally in the slave controller except the SD Status Register (SSR). The SD status register contains 512 bits of mostly read only data so it is more appropriate to be stored in the user logic.

When the host issues a command to read the SD Status Register, the controller forwards the read request to the user logic through a regular burst read transfer. with CS#=”01” and Address = 08000000(hex) to indicate SD Status Register read. Data is transfer at the assertion of RDY# as in normal data read.

The SIZE signal is always 010(hex) for SSR read to indicate 512 bits (16 words) transfer.

Bit 511 and 510 of the SSR are generated internally by the SD slave controller because it contains information known to the slave controller. When reading the last word of the SSR, the user can provide either “1” or “0” in this two bits and both will be substituted by the slave controller when forward to the host.

2.21 CSA Access

This section applies to SDIO only and is not applicable to SD memory.

SDIO function contains the CSA address space which is mapped to IO function 0. The CSA address space can be read or write by the user interface. The user interface bus protocol is the same as in regular access with In_CS#=”01” and Address[31:24] = 0x00 to indicate CSA access. Address bit 23:0 are the address

within the CSA space. The access can be either read or write as indicated by In_WR. Data is transfer at the assertion of In_RDY# as in normal data read and write.

The time limits for CSA access are the same as SDIO transfers described in Section 2.4, “Read Access Time Limit” on page 15 and Section 2.8, “Write Access Time Limit” on page 18.

2.22 CSD Writable Bits

This section applies to SD memory only and is not applicable to SDIO.

The CSD contains a combination of read-only bits, multiple time writable bits, and writable-once bits. The read-only bits are provided to the SD slave controller by static inputs. The multiple time writable bits are stored within the SD slave controller. The multiple time writable bits can be broken down into two components: temporary write protect bit, and CSD CRC. The temporary write protect bit is maintained completely by the SD slave controller. The CSD CRC is stored in the user logic. The CSD CRC corresponds to bits 7 to 1 of the CSD static input.

The writable-once bits are provided through static inputs by the user as well. A writable-once bit in the CSD is a bit that can only be written to once during the lifetime of the card. The user logic is responsible for keeping track of whether or not the bit has been written. The user logic must also ensure that once these bits are written to, the value is maintain forever. The number of writable once bits differs depending on the version of CSD is implemented. In version 1.0 bits 15, 14, 13, 11, and 10 are writable once. In version 2.0 only bits 14 and 13 are writable once, with bits 15, 12, and 10 being fixed values implemented by the SD slave controller.

To write CSD data to the user logic, ADS# is asserted with CS#="01", address = 08008090 (hex) and WR="1". The data to be written is on the DWR signal. The user logic must ensure that newly written values appear on the CSD static input to the SD slave controller. The bits to be written are described above.

2.23 SDIO Status Signals

When the SD card indicates that a function has been enabled, the user begins initialization for this function. The enable signals for the 7 SDIO functions are located in the CCCR at address 0x02, with bit 7 to 1 mapping to functions 7 to 1. When the enabled function has finished initialization and is ready, the host shall signal that the function is ready. The ready signals for the 7 SDIO functions are located in the CCCR at address 0x03, with bits 7 to 1 mapping to functions 7 to 1.

Interrupts are available to SDIO functions and may be sent anytime by the user by asserting the interrupt bit that corresponds to the function. The interrupt bits are

located in the CCCR at address 0x05, with bits 7 to 1 corresponding to functions 7 to 1. This request is then forwarded to the host by the SD slave controller.

2.24 High Speed Mode

The card can be switched between high speed mode and default speed mode in two different ways, one for each mode of operation, SD Memory and SDIO.

For SD Memory, the switch to high speed mode is carried out using the two command sequence switch function as described in section Section 2.19, “Switch Function” on page 26. The first command, which is the write, contains the request for in the data block sent on DWR. The second command, which is the read request for the status block, contains whether the switch is successful within the data block.

For SDIO, the host writes simply writes to an internal register to switch to high speed mode. The content of the internal register is available to the user logic on bit 153 of CCCR_OUT. In order for the write to be successful, high-speed mode must be supported. This support is indicated in the CCCR address 0x13 bit 0. This bit is part of the static input provided by the user logic on the CCCR signal bit 152. For more details see Section 4.7, “CCCR Register” on page 41.

In an SD Combo card, either of the two methods can be used and both the SD Memory and SDIO portions of the card switches to high speed mode.

2.25 Reset

The controller core should be reset by the RESET# pin during initial power up. In addition, soft reset can be executed by the host by sending a command to the SD Slave. The soft reset is forwarded by the SD Slave to the user logic by asserting either the SRESETI# or SRESETM# signal. Soft reset commands are card type specific (i.e. SD Memory or SDIO). SRESETI# indicates SDIO reset and SRESETM# indicates SD memory reset.

2.26 Initialization

During initial power up of SDIO device, the user logic may require a long time to reach operating stage. The SDIO_CRD_RDY signal can be used by the user logic to indicate its readiness. It is asserted by the user logic so that the proper response can be sent to the host by slave controller to indicate the status of the user logic. For SD Memory device, this is done through bit 31 of the OCR input to the controller core.

2.27 Static Inputs

Except for SD Status Register, all the other registers are contained internally in the slave controller. A number of read-only register data such as Manufacturer ID is provided to the SD Slave via static inputs which can be hardwired by the user to the proper value. Some registers are completely self-contained within the controller core and do not require static inputs. For each register requiring static value from the user, there is a dedicated static input signal. The bit index to the static input matches the bit index of the corresponding bit in the register.

Most registers contain a mixture of user provided signals, reserved bits and contents generated from within the SD Slave controller. In this case, the reserved bits and SD slave controller generated bits will be forwarded to the SD host. The corresponding bit slice on the static input shall be tied to either “1” or “0” and will be ignored by the controller. For more details about the SD Registers, see section 4.

3 SD Interface

This section describes the function of the SD slave controller from the perspective of the SD interface. On the SD bus, the SD Host and the SD Slave Controller communicate with each other via the 9-pin SD Bus. Based on the information received on the SD bus, the slave controller interacts with the user logic via the user interface. The SD Slave controller in combination with a user device can used to create a wide variety of products from memory cards to wireless network cards.

The CMD and DT signals are synchronized with the SDCLK clock signal. If the card is operating in high speed mode, outputs are generated from the rising edge of SDCLK. If the card is operating in the low speed mode, outputs are generated from the falling edge of SDCLK. In both cases, inputs are sampled at the rising edge of SDCLK.

3.1 Compliance with the latest versions

The SD Slave controller is compliant with the latest versions of the SD Memory Memory and SDIO specifications 2.0. Version 2.0 of the SD Memory defines two types of memory cards, standard capacity cards and high capacity cards. The former supports sizes up to 2GB, while the latter supports sizes from 4GB to 32GB. Standard capacity cards work properly with any SD Host, while a higher capacity card requires a high capacity card compliant host.

3.2 Modes of operation and card type

The SD Slave controller supports both the SD and SPI bus protocols. In SD, data can be transferred across all 4 bits of the DT line or across just 1 line. 8-bit mode is supported if the optional MMC feature is included. In SPI only 1-bit wide DT line is supported. The bus protocol used by the host is automatically detected by the SD Slave controller. The differences in protocol are handled completely by the SD Slave controller. Signals along the user interface remains the same no matter which protocol is being used.

The SD Slave controller can be used in a device to create an SD Memory card, SDIO card, or a combo card, which contains both memory and IO components. SD memory and each SDIO function has its own user interface so they can be operated in parallel. The bus protocol is the same for all user interfaces. With the great flexibility, the controller can be combined to create a wide variety of SD devices.

3.3 Command and Response

Any task is initiated by the host issuing a command to the SD Slave controller. Any SD Memory and SDIO command presented in the SD Specification can be processed by the controller. The command is shifted into and processed by the controller where it is decoded and a CRC check is performed to verify data integrity by the controller's CRC verifier. Depending on the command type, the command may be processed internally with no user logic interaction or forward the request to the user logic on the simple user interface. The requests forwarded to the user logic include read and write operations. Once the command has been processed, a response is sent back to the host. A CRC, generated by the controller's CRC generator, is appended to the response and sent back to the host. The reception of the command and the generation of the response is handled completely in the SD slave controller.

3.4 Data Transaction

Most interaction between the SD slave controller and the user interface is in the form of a read or write data transfer. A read or write command is received by the SD Slave controller from the SD Host. The command is forwarded to the user along the simple address/data request.

For a write operation, data is transferred across the DT signal and shifted in by the SD Slave controller. The data is unpacked by the SD Slave controller, and stored into the data buffer. Once the data has been collected into the buffer, the CRC is verified before a request is sent out to the user. If the CRC check fails, a request is never sent to the user logic, ensuring that only valid data is sent to the user. Once an acknowledgement signal is sent, the data is transferred.

For a read operation, a data is sent by the SD slave controller to the user. A response is sent by the user along with the data requested. The data is stored in the data buffer until a data block is received and then transfer is terminated. The data is then packaged with overhead bits and a CRC that is generated by the SD slave controller's CRC generator. This data is then shifted out on the DT signal to the system host.

In both cases, the responsibility of the user logic is to send or receive the raw data requested by the SD Slave controller. The number of blocks, timing of the data transfer along the SD interface, and the generation/verification of the CRC is all handled within the SD slave controller, simplifying the user interface to a simple address/data interface.

3.5 Buffer

The SD Slave Controller features a data buffer to store data for transfer between

the SD DT line and the user interface. Each user interface (function) has its dedicated data buffer. This buffer is used for both read and write operations. The buffer can hold up to 1024 32-bit words, which is 4096 bytes. Larger buffer size is implemented if the optional MMC feature is included. Smaller buffer size can be configured upon user request.

The maximum block size supported by the SD Slave controller is 2048 bytes, which is 512 32-bit words. Thus, even in the worst case of the largest block size, the next block can be written into the buffer, while the first is being sent, leading to an increase in performance. Because data transfer timing is handled by the SD Slave controller, the user logic can send data and receive data at its own pace and insert wait states when needed.

3.6 Stop and Data Termination

The number of blocks in a multiple block data transfer is not specified before transfer in most of the transfers (it is possible to specify one in an SDIO data transfer, but not necessary). The SD mechanism to signal the end of a transfer to the SD Slave controller is a stop command. The command is processed as discussed above and a response sent. In effect, no more data requests are sent to the user logic by the SD slave controller. The stop command is processed by the slave controller. As a result of a stop read command, the core issue the abort transaction by the ABORT# signal.

3.7 SD Registers

All the SD registers are implemented within the SD Slave controller, except the SD Status Register. The optional Driver Stage Register (DSR) is also exists in the SD Slave controller. Some of the register contents are device specific, such as manufacturer ID, and is provided to the SD Slave controller by the user logic via static inputs. In the case of the SD Status Register, its size was too great to be completely supported by static inputs and thus is implemented in the user logic and its contents transferred by data transfer as described above. Aside from the static inputs and the SD Status Register, all register related transactions are handled entirely by the SD Slave controller. Because of the simplicity of the user interface, a read access to the SSR on the user interface is almost the same as a read access. This simplicity minimizes design requirements by the user logic.

3.8 Busy

The SD Slave controller can assert a busy signal to the system host by holding the DT[0] line low. This is the method by which the SD Slave controller controls traffic along the SD interface. The busy signal is asserted when the internal buffer

cannot store an additional block or if the card is programming. Because the traffic management is handled within the SD Slave controller, data transfer can occur normally as described above without concern of the SD interface. The traffic management burden is completely on the SD Slave controller, thus reducing the design requirements by the user logic.

3.9 SDIO Interrupts

An SDIO card may signal an interrupt to the host by pulling the DT[1] line low. A card operating in 1-bit mode can signal an interrupt at anytime because only DT[0] is used for data transfer. In 4-bit mode, an interrupt is allowed to be signaled only during a period of time known as the interrupt period.

3.10 Read-Wait

The read-wait feature allows the host to stall a read data block transfer. The host does so by pulling DT[2] low at a designated time between the end bit of a read block and before the start bit of the next read block. While DT[2] is low, the SD Slave does not transfer any read data and the host may issue commands that do not use the DT line to other functions. When the host has finished, the read data transfer is resumed by the host releasing DT[2].

3.11 Suspend and Resume

The SD Slave controller supports the optional suspend and resume. It can suspend a transaction, such as a multi-block transfer, allow the system host to issue other commands, and resume the suspended transaction. Suspend and resume is only supported in SDIO. A transaction can only be suspended only when the DT line is not transferring data. In the multi-block case, a transaction can be suspended between data blocks. Because complete blocks are still being read and written the user logic need not be aware of a suspended transaction. As before, data is sent or received as the requests come in by the SD slave controller.

4 SD Registers

The SD controller contains most of the required SD registers. Access to these registers is processed by the SD slave controller automatically and the access is not forwarded to the user interface. The SD Status Register (SSR) is the only exception. Because this register contains 512 read only bits, it is more efficient that these read only data bits are stored in the user logic and provided to the slave controller through the register read command described in previous section.

The following table lists all the SD registers. The CSR, DSR and RCA is completely internal to the controller core and is not dependent on as static inputs from the user. Other register except for SD Status use some of the value from the static inputs provided by the user.

Table 10: SD Registers

Register	Source
OCR	Internal and static input
CID	Internal and static input
CSD	Internal and static input
SCR	Internal and static input
SD Status	User Logic
SDIO OCR	Internal and static input
CCCR	Internal and static input
FBR	Internal and static input. One for each SDIO function from 1 to 7
CSR	Internal to controller core
DSR	Internal to controller core
RCA	Internal to controller core

4.1 OCR Register

The OCR consists of 32-bits of read only values. The read only value are static input to the core through the OCR_MEM inputs. The following table lists the bits

in this register for SD Memory. This register is modified for MMC. Please see the Appendix on MMC option for detail.

Table 11: OCR Register

Bits	Description	Source
31	Card power up status bit	Input from OCR_MEM
30	Card capacity status	Input from OCR_MEM
29:24	Reserved	Generated by the controller
23:15	VDD voltage	Input from OCR_MEM
14:8	Reserved	Generated by the controller
7	Low voltage range	Input from OCR_MEM
6:0	Reserved	Generated by the controller

4.2 CID Register

The CID consists of 128-bits of read only values. The read only value are static input to the core through the CID inputs. The following table lists the bits in this register. This register is modified for MMC. Please see the Appendix on MMC option for detail.

Table 12: CID Register

Bits	Description	Source
127:120	Manufacturer ID	From CID input signal
119:104	OEM/Application ID	From CID input signal
103:64	Product name	From CID input signal
63:56	Product revision	From CID input signal
55:24	Product serial number	From CID input signal
23:20	Reserved	Generated by the controller
19:8	Manufacturing date	From CID input signal
7:1	CRC7 checksum	From CID input signal

Table 12: CID Register

Bits	Description	Source
0	Reserved	Generated by the controller

4.3 CSD Version 1.0 Register

This register has 128 bits and is needed for version 1.0 SD card. The read only value are static input to the core through the CSD inputs. The following table lists the bits in this register. This register is modified for MMC. Please see the Appendix on MMC option for detail.

Table 13: CSD Version 1.0 Register

Bits	Description	Source
127:126	CSD structure To indicate version 1.0, these two bits should be “00”	From CSD input signal
125:120	Reserved	Generated by the controller
119:112	TAAC	From CSD input signal
111:104	NSAC	From CSD input signal
103:96	TRAN_SPEED	From CSD input signal
95:84	CCC	From CSD input signal
83:80	READ_BL_LEN	From CSD input signal
79	READ_BL_PARTIAL	From CSD input signal
78	WRITE_BLK_MISALIGN	From CSD input signal
77	READ_BLK_MISALIGN	From CSD input signal
76	DSR_IMP	From CSD input signal
75:74	Reserved	Generated by the controller
73:62	C_SIZE	From CSD input signal
61:59	VDD_R_CURR_MIN	From CSD input signal
58:56	VDD_R_CURR_MAX	From CSD input signal

Table 13: CSD Version 1.0 Register

Bits	Description	Source
55:53	VDD_W_CURR_MIN	From CSD input signal
52:50	VDD_W_CURR_MAX	From CSD input signal
49:47	C_SIZE_MULT	From CSD input signal
46	ERASE_BLK_EN	From CSD input signal
45:39	SECTOR_SIZE	From CSD input signal
38:32	WP_GRP_SIZE	From CSD input signal
31	WP_GRP_ENABLE	From CSD input signal
30:29	Reserved	Generated by the controller
28:26	R2W_FACTOR	From CSD input signal
25:22	WRITE_BL_LEN	From CSD input signal
21	WRITE_BL_PARTIAL	From CSD input signal
20:16	Reserved	Generated by the controller
15	FILE_FORMAT_GRP	From CSD input signal
14	COPY	From CSD input signal
13	PERM_WRITE_PROTECT	From CSD input signal
12	TMP_WRITE_PROTECT	Generated by the controller
11:10	FILE_FORMAT	From CSD input signal
9:8	Reserved	Generated by the controller
7:1	CRC	From CSD input signal
0	Reserved	Generated by the controller

4.4 CSD Version 2.0 Register

This register has 128 bits and is needed for high capacity (version 2.0) SD card. The read only value are static input to the core through the CSD inputs. The fol-

lowing table lists the bits in this register.

Table 14: CSD Version 2.0 Register

Bits	Description	Source
127:126	CSD structure To indicate version 2.0, these two bits should be “01”.	Generated by the controller
125:120	Reserved	Generated by the controller
119:112	TAAC	Generated by the controller
111:104	NSAC	Generated by the controller
103:96	TRAN_SPEED	From CSD input signal
95:84	CCC	From CSD input signal
83:80	READ_BL_LEN	Generated by the controller
79	READ_BL_PARTIAL	Generated by the controller
78	WRITE_BLK_MISALIGN	Generated by the controller
77	READ_BLK_MISALIGN	Generated by the controller
76	DSR_IMP	From CSD input signal
75:70	Reserved	Generated by the controller
69:48	C_SIZE	From CSD input signal
47	Reserved	Generated by the controller
46	ERASE_BLK_EN	Generated by the controller
45:39	SECTOR_SIZE	Generated by the controller
38:32	WP_GRP_SIZE	Generated by the controller
31	WP_GRP_ENABLE	Generated by the controller
30:29	Reserved	Generated by the controller
28:26	R2W_FACTOR	Generated by the controller
25:22	WRITE_BL_LEN	Generated by the controller
21	WRITE_BL_PARTIAL	Generated by the controller

Table 14: CSD Version 2.0 Register

Bits	Description	Source
20:16	Reserved	Generated by the controller
15	FILE_FORMAT_GRP	Generated by the controller
14	COPY	From CSD input signal
13	PERM_WRITE_PROTECT	From CSD input signal
12	TMP_WRITE_PROTECT	Generated by the controller
11:10	FILE_FORMAT	Generated by the controller
9:8	Reserved	Generated by the controller
7:1	CRC	From CSD input signal
0	Reserved	Generated by the controller

4.5 SCR Register

This register has 64 bits. The read only value are static input to the core through the SCR inputs. The following table lists the bits in this register.

Table 15: SCR Register

Bits	Description	Source
63:60	SCR_STRUCTURE	From SCR input signal
59:56	SD_SPEC	From SCR input signal
55	DATA_STAT_AFTER_ERASE	From SCR input signal
54:52	SD_SECURITY	From SCR input signal
51:48	SD_BUS_WIDTHS	From SCR input signal
47:32	Reserved	Generated by the controller
31:0	Reserved for Manufacturer usage	From SCR input signal

4.6 SDIO OCR Register

This register has 24 bits. The read only value are static input to the core through the SDIO_OCR inputs. The following table lists the bits in this register.

Table 16: SDIO OCR Register

Bits	Description	Source
23:8	VDD voltage	From SDIO_OCR input signal
7:0	Reserved	Generated by the controller

4.7 CCCR Register

This register has 256 bytes. The read only value are static input to the core through the CCCR and the CCCR_UPPER inputs. The following table lists the bits in this register.

Table 17: CCCR Register

Bits	Description	Source
2047:1920	Manufacturer specific	From CCCR_UPPER input signal
1919:154	Reserved	Generated by the controller
153	EHS	Generated by the controller
152	SHS	From CCCR input signal
151:146	Reserved	Generated by the controller
145	EMPC	Generated by the controller
144	SMPC	From CCCR input signal
143:128	IO block size for function 0	Generated by the controller
127:120	Ready flags	Generated by the controller
119:112	Exec flags	Generated by the controller
111	DF	Generated by the controller
110:108	Reserved	Generated by the controller

Table 17: CCCR Register

Bits	Description	Source
107:104	Function select	Generated by the controller
103:98	Reserved	Generated by the controller
97	BR	Generated by the controller
96	BS	Generated by the controller
95:72	CIS pointer	From CCCR input signal
71:64	Card capability	From CCCR input signal except bit 69 which is generated by the controller
63	CD disable	Generated by the controller
62	SCSI	From CCCR input signal
61	ECSI	Generated by the controller
60:58	Reserved	Generated by the controller
57:56	bus width	Generated by the controller
55:52	Reserved	Generated by the controller
51	RES	Generated by the controller
50:48	AS2:0	Generated by the controller
47:41	Interrupt pending	From CCCR input signal
40	Reserved	Generated by the controller
39:32	Interrupt enable	Generated by the controller
31:25	IO ready	From CCCR input signal
24	Reserved	Generated by the controller
23:17	IO enable	Generated by the controller
16:12	Reserved	Generated by the controller
11:8	SD specification revision	From CCCR input signal
7:0	CCCR/SDIO revision	From CCCR input signal

4.8 FBR Register

This register has 256 bytes. There is one FBR register for each function for function 1 to 7. The read only value are static input to the core through the FBR1 inputs. If multiple SDIO function is implemented, additional registers and inputs (FBR2, FBR3 ... FBR7) will be added. The following table lists the bits in this register.

Table 18: FBR Register

Bits	Description	Source
2047:144	Reserved	Generated by the controller
143:128	IO block size for function 1	Generated by the controller
127:120	CSA window	Generated by the controller
119: 96	CSA pointer	Generated by the controller
95:72	CIS pointer	From FBR1 input signal
71:18	Reserved	Generated by the controller
17	EPS	Generated by the controller
16	SPS	From FBR1 input signal
15:8	Extended standard SDIO Function interface code	From FBR1 input signal
7	CSA enable	Generated by the controller
6	CSA support	From FBR1 input signal
5:4	Reserved	Generated by the controller
3:0	Standard SDIO Function interface code	From FBR1 input signal

Appendix A. Optional AHB Interface

This appendix describes optional AHB bus interface. When this option is used, an AHB bus replaces each of the generic user interface ports described above. This section first describes the basic AHB bus protocol. Once this protocol is understood, it is easy to see how the AHB bus replaces the standard user interface for read and write access to the user logic. Access specific to the slave controller such as erase and programming are described in the last section of the appendix.

The AHB bus is designed with a central arbitration unit to control routing of different signals between multiple masters and slaves. Each master or slave device receives dedicated signals to and from the arbitration unit with no tri-state signals. The following diagram illustrates typical bus architecture.

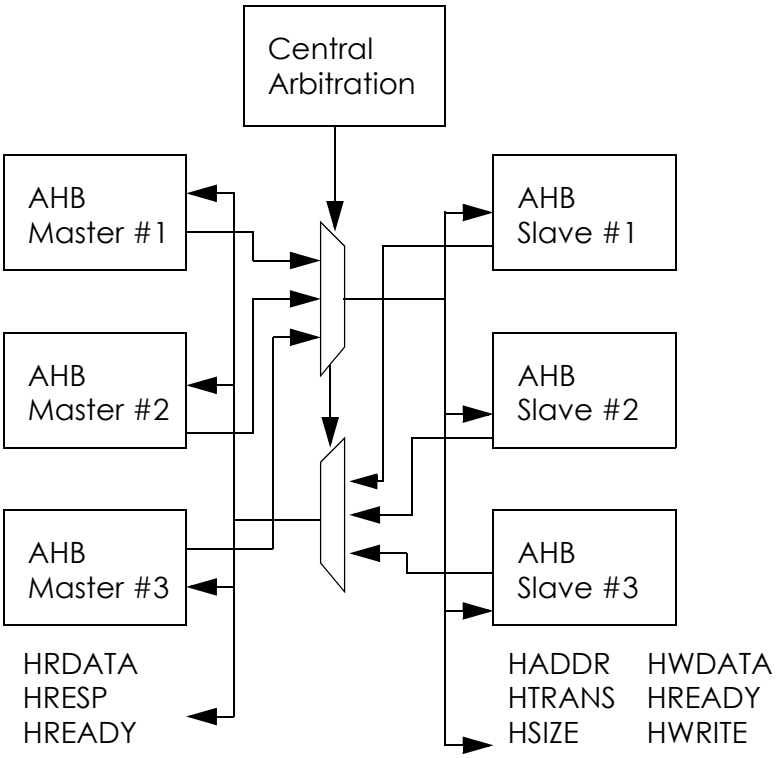


Figure A.1 AHB Bus System

A.1 Basic Transfer

In this section, it is important to remember that the SDIO slave controller functions as an AHB bus master while the user logic functions as an AHB bus slave.

An AHB transfer contains the address phase and data phase. For single transfer, only one data phase follows an address phase. For burst transfer, multiple data phase follows an address phase.

The master initiates a transfer by driving the address and control signals after the rising clock edge. It is sampled by the bus slave at the following rising clock edge to complete the address phase.

The address phase may overlap the data phase of a previous data transfer. However, an address phase is valid only when it occurs at the same time or after the last data phase of a previous transfer. Each bus slave uses the HREADY input to qualify an address. This will be described in more detail later.

The data phase timing is controlled by the bus slave being access. The bus slave may de-assert the HREADY signal to insert wait state in conjunction with the HRESP signals to indicate different types of responses. The following diagram shows a basic transfer.

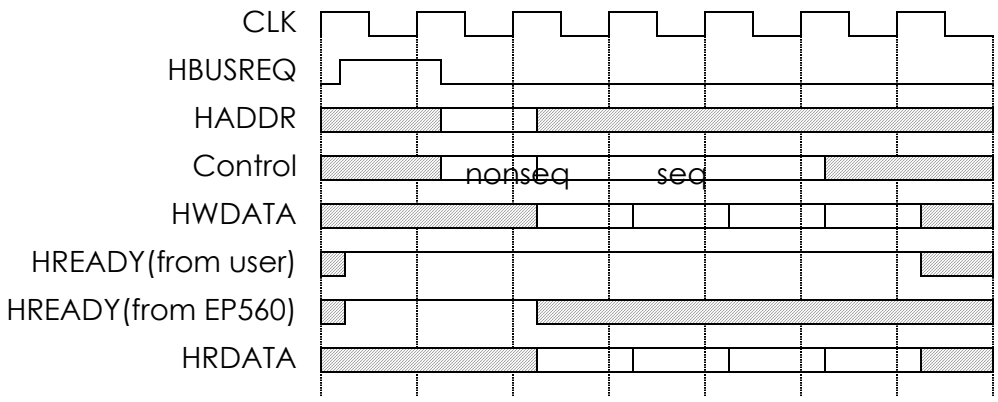


Figure A.2 Transfer with no wait state

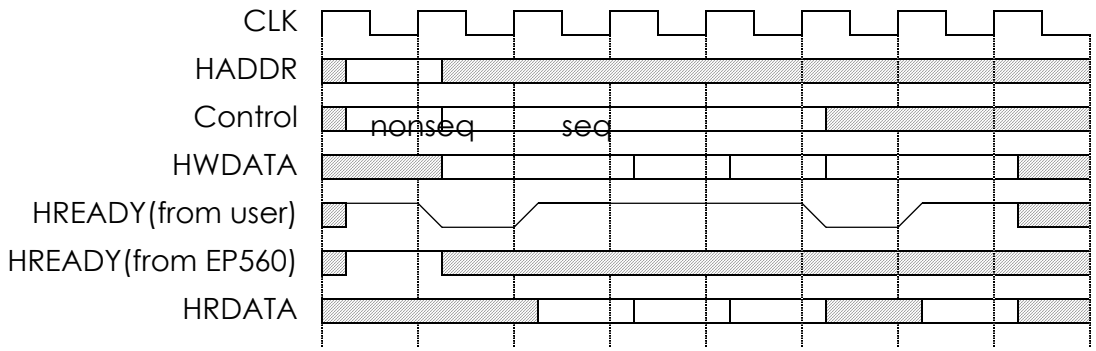


Figure A.3 Transfer with wait state

A.2 Transfer type

The master uses the transfer type signals (HTRANS) to indicate the bus access. The HTRANS signal is driven by the current bus master and is encoded with the following combinations:

Table 19: Transfer Types

HTRANS[1:0]	Description
0 0	IDLE Indicates that no data transfer is required. The IDLE transfer type is used when a bus master is granted the bus, but does not wish to perform a data transfer. Bus slaves must always provide a zero wait state OKAY response to IDLE transfers and the transfer should be ignored by the bus slave.

Table 19: Transfer Types

HTRANS[1:0]	Description
0 1	BUSY The BUSY transfer type allows bus masters to insert wait state cycles in the middle of bursts of transfers. This transfer type indicates that the bus master is continuing with a burst of transfers, but the next transfer cannot take place immediately. When a master uses the BUSY transfer type the address and control signals must reflect the next transfer in the burst. The transfer should be ignored by the bus slave. Slaves must always provide a zero wait state OKAY response, in the same way that they respond to IDLE transfers.
1 0	NONSEQ Indicates the first transfer of a burst or a single transfer. The address and control signals are unrelated to the previous transfer. Single transfers on the bus are treated as bursts of one and therefore the transfer type is NONSEQUENTIAL.
1 1	SEQ The remaining transfers in a burst are SEQUENTIAL and the address is related to the previous transfer. The control information is identical to the previous transfer. The address is equal to the address of the previous transfer plus the size (in bytes). In the case of a wrapping burst the address of the transfer wraps at the address boundary equal to the size (in bytes) multiplied by the number of beats in the transfer (either 4, 8 or 16).

The following diagram shows the usage of the 4 transfer types. At cycle 1, the master is not ready to issue any commands so the transfer type of IDLE is issued. Cycle 2 is the beginning of an access so NONSEQ is issued. It is a single access and the master issue another access at cycle 3 using NONSEQ transfer type again. Since this access is a burst data access, the master issues SEQ in cycle 4 to continue the transfer. Wait state is inserted by the master at cycle 5 and 6 with the BUSY transfer type. Finally two more SEQ transfer type at cycle 7 and 8 com-

pletes the data phase of this burst access for four data.

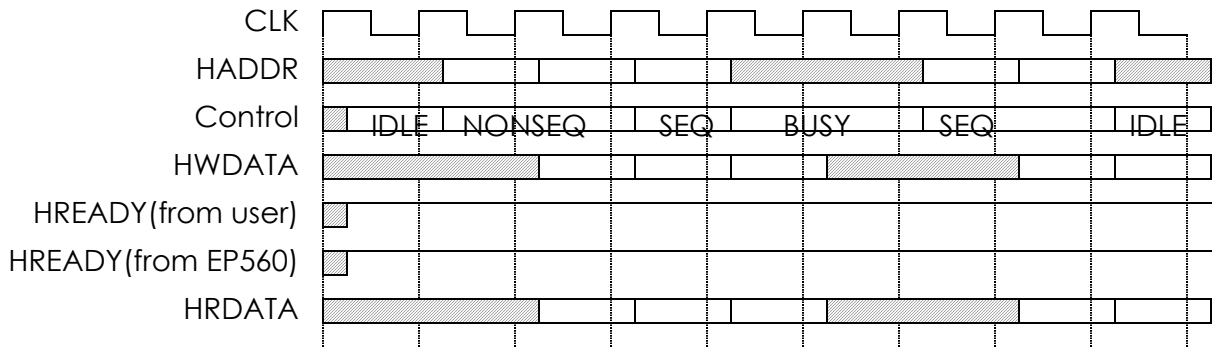


Figure A.4 Transfer Types

A.3 Qualified Address Phase

A master may issue an address phase overlapping with the last data phase of a previous transfer. If the bus slave of the previous transfer does not insert wait state for the data phase, the data phase terminates at the same as the address phase for the new transfer. In this case the data phase for the next transfer follows immediately. If the bus slave of the previous transfer inserts wait state for the last data phase, the address phase for the new transfer has to be extended by the master until the previous data phase terminates.

Both the master and the bus slave for the new transfer samples the HREADY input to qualify the address phase. If HREADY is active at the time of the address phase, the address phase is qualified. If HREADY is inactive at the time of the address phase, the master must extend the address phase and the bus slave not respond to it. The system arbiter is responsible to merge all HREADY signals from different bus slaves and generate HREADY to all master and bus slave devices. By observing the HREADY input, a bus slave can determine if a new

address phase is valid or not.

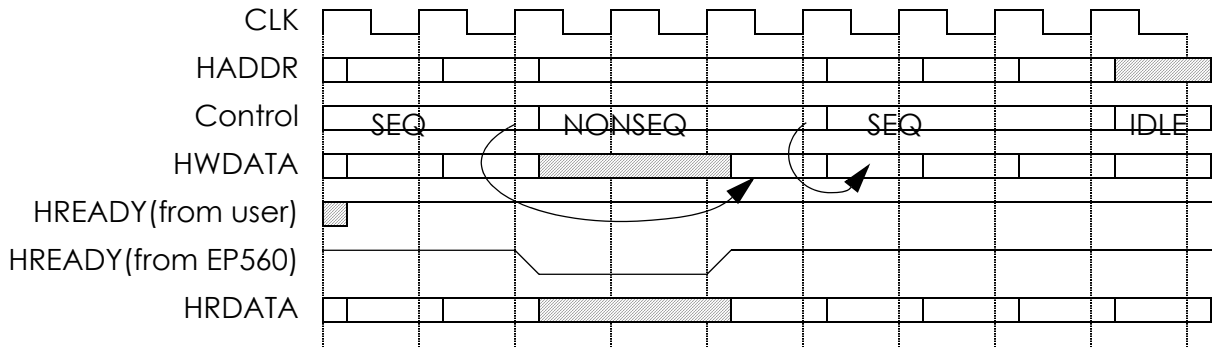


Figure A.5 NONSEQ Address phase delayed by HREADY

A.4 Bus Slave Response to Data Phase

Data phase of a transfer is controlled both by the master and by the bus slave. This section will describe the bus slave's control to data transfer. Master can also control the timing of data transfer by inserting the BUSY cycle. This will be described in later sections.

Once a bus slave receives a qualified address phase, it starts to process the data transfer. For write access, the write data is provided by the master one cycle after the qualified address phase. The write data is driven on the HWDATA bus until the bus slave asserts HREADY signal. The system arbiter is responsible to route the HREADY signal from this bus slave to the master.

If the bus slave cannot complete the write operation immediately, it can insert wait state by de-asserting the HREADY signal for one or more cycles until it is ready to complete the transfer. The master is required to continue to drive the valid data on the HWDATA bus during the wait state. For burst data transfer, the master will issue the SEQ transfer type after the qualified address phase. The next address is driven by the master on the HADDR bus and the bus slave will continue the data transfer using the HREADY signal.

Read data transfer is handled by the HREADY signal similar to write transfer. Wait state can be inserted by the bus slave until it drives the valid data on the

HRDATA bus.

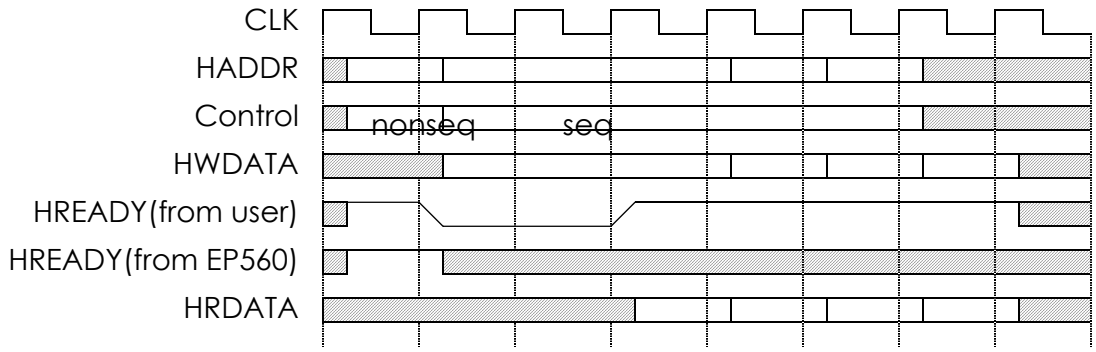


Figure A.6 Read/write transfer with wait state

A.5 Burst Data Transfer

The AHB bus supports transfer size of four, eight and sixteen-beat and undefined-length bursts and single transfers. Both incrementing and wrapping bursts are supported in the protocol:

• Incrementing bursts access sequential locations and the address of each transfer in the burst is just a linear increment of the previous address.

• Wrapping bursts access location as if the data are within one cache line without crossing the cache line boundary. If the start address of the transfer is not aligned to the total number of bytes in the burst (size x beats) then the address of the transfers in the burst will wrap when the boundary is reached. For example, a four-beat wrapping burst of word (4-byte) accesses will wrap at 16-byte boundaries. Therefore, if the start address of the transfer is 0x34, then it consists of four transfers to addresses 0x34, 0x38, 0x3C and 0x30.

Bursts must not cross a 1kB address boundary. It is the responsibility of the master not to attempt to start a fixed-length incrementing burst which would cross this boundary.

It is acceptable to perform single transfers using an unspecified-length incrementing burst which only has a burst of length one.

The following table list the allowed burst transfer size:

Table 20: Burst Transfer Size

HBURST[2:0]	Type	Description
000	SINGLE	Single transfer
001	INCR	Incrementing burst of unspecified length
010	WRAP4	4-beat wrapping burst
011	INCR4	4-beat incrementing burst
100	WRAP8	8-beat wrapping burst
101	INCR8	8-beat incrementing burst
110	WRAP16	16-beat wrapping burst
111	INCR16	16-beat incrementing burst

A.6 Data Size

The data bus width is 32-bit but the master has the optional to transfer data with smaller data width. The HSIZE and the HBURST input together determine the data transfer size. Total data transfer equals data size times burst size.

Table 21: Data Size Encoding

HSIZE[2:0]	Description
000	Byte
001	Halfword (16-bit)
010	Word (32-bit)
Others	Reserved

The HSIZE signal supports controls the transfer size and the byte lane used for

each transfer is according to the following table:

Table 22: Byte Lane Assignments

Width	Starting address	Data [31:24]	Data [23:16]	Data [15:8]	Data [7:0]
byte	0				X
byte	1			X	
byte	2		X		
byte	3	X			
halfword	0			X	X
halfword	2	X	X		
word	0	X	X	X	X

A.7 Retry and Bus Slave Response

Previous section described the bus slave's responses to transfer with the HREADY signal. In addition the HREADY signal, the bus slave also provide the response type using the HRESP signals to further indicate its response types:

Table 23: Bus Slave Responses

HRESP[1:0]	Type	Description
0 0	OKAY	When HREADY is high this shows the transfer has completed successfully. The OKAY response is also used for any additional cycles that are inserted, with HREADY low, prior to giving one of the three other responses.
0 1	ERROR	This response shows an error has occurred. The error condition should be signalled to the bus master so that it is aware the transfer has been unsuccessful. A two-cycle response is required for an error condition.

Table 23: Bus Slave Responses

HRESP[1:0]	Type	Description
1 0	RETRY	The RETRY response shows the transfer has not yet completed, so the bus master should retry the transfer. The master should continue to retry the transfer until it completes. A two-cycle RETRY response is required.
1 1	SPLIT	The transfer has not yet completed successfully. The bus master must retry the transfer when it is next granted access to the bus. The bus slave will request access to the bus on behalf of the master when the transfer can complete. A two-cycle SPLIT response is required.

The ERROR, RETY and SPLIT response required at least two cycles to process. The bus slave is required to insert wait state prior to signaling any one of the three responses. The following timing diagram shows a retry response.

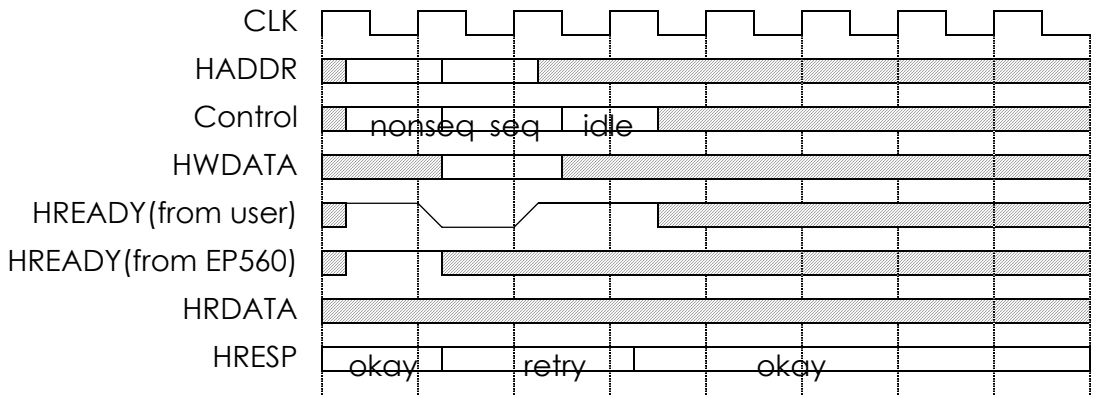


Figure A.7 Retry

A.8 Master Wait State

The BUSY transfer type allows bus masters to insert wait states in the middle of bursts of transfers. This transfer type indicates that the bus master is continuing with a burst of transfers, but the next transfer cannot take place immediately. When a master uses the BUSY transfer type the address and control signals must reflect the next transfer in the burst. The transfer should be ignored by the bus slave. Bus slaves must always provide a zero wait state OKAY response, in the same way that they respond to IDLE transfers.

The following diagram shows the master insert wait state in the middle of a burst transfer by using the BUSY transfer type.

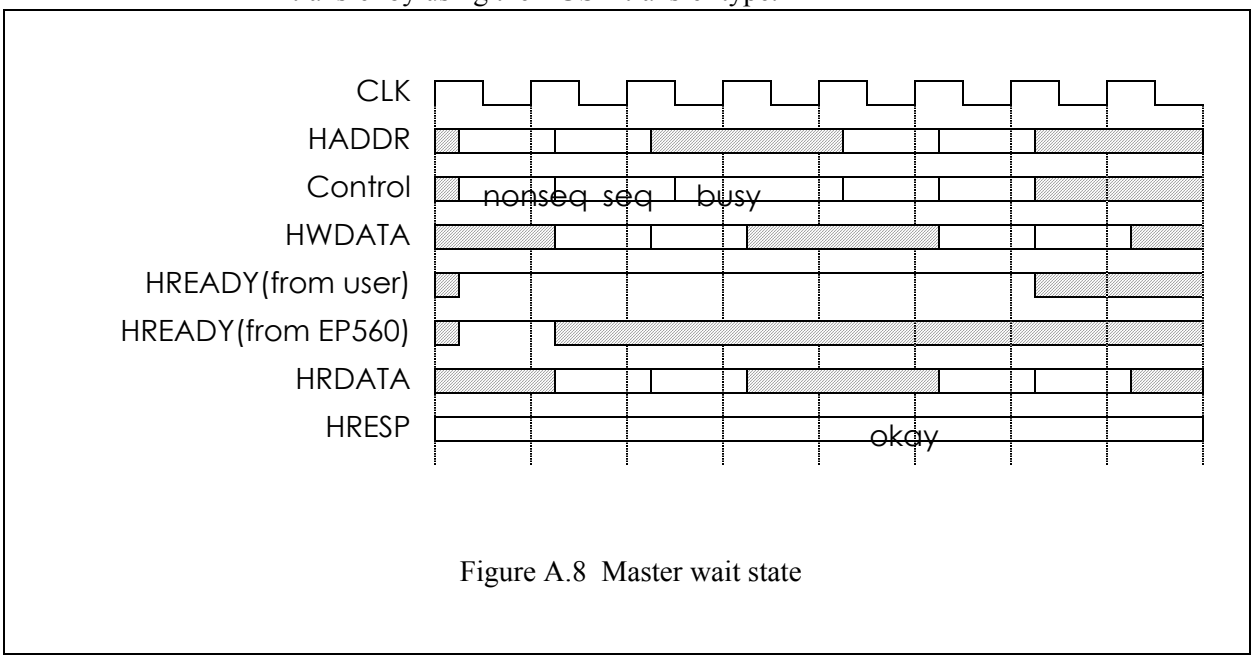


Figure A.8 Master wait state

A.9 Early Burst Termination

The master can terminate a burst transfer before all the data transfer are completed. The bus slave can determine when a burst has terminated early by monitoring the HTRANS signals and ensuring that after the start of the burst every transfer is labelled as SEQ or BUSY. If a NONSEQ or IDLE transfer occurs then this indicates that a new burst has started and therefore the previous one must have

been terminated.

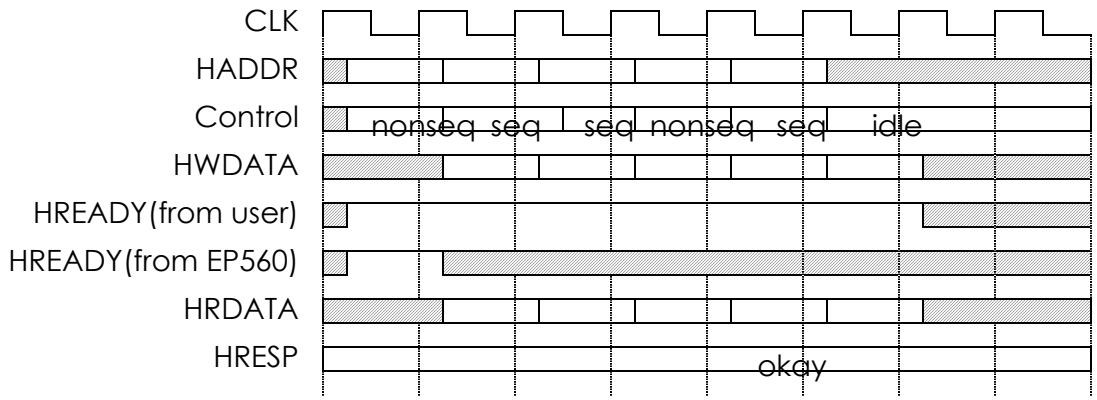


Figure A.9 Burst terminated by NONSEQ or IDLE transfer

A.10 Access Address

Instead of using the CS# signal, the AHB bus uses the HSEL signal to indicate the access address space. The access address contains additional information according to the HSEL signal.

Table 24: Access Address

HSEL[1:0]	Address(HADDR)	Note
01	All 32-bit are specified according to the SD command	Normal SD memory or SDIO access
10	As specified in generic user interface	Special access

A.11 Erase Command

Erase command are issued to the AHB bus as write access with specific address. The user logic can response with normal completion (OKAY response) if the erase commands result in no error. The user logic must respond to Erase start address

and Erase end address with ERROR response if the address range is not valid. On Erase execution command, the user logic must respond with ERROR if the some erase block is skipped due to write protection.

There are additional timing requirement for all erase commands. Like all AHB bus requests, the bus master first asserts HBUSREQ signal. It then waits for HGRANT before issuing the request on AHB bus. From the time HBUSREQ is issued for erase command, the core must receive bus grant and receives the user logic's response to the command (OKAY or ERROR) within 50 clock cycles.

A.12 Write Response

User logic can use ERROR response to indicate write protection violation for SR memory and IO read. When write protection error occurs, it triggers the following events:

1. The bus transfer on the user interface terminates immediately. No more data will be transfer even if it is a burst write.
2. The core sets the WP_VIOLATION_ERROR bit (bit 26) of the Card Status Register (CSR).
3. If it is a multiple block write, remaining data blocks already received by the core will not be written to the user interface and buffered data will be discarded.
4. If the SD interface continue to shift in write data, the core will respond with no ECC status (status="111" with no start bit) to indicate write error.

A.13 Programming Command

After all the data associated with one SD write command has been sent out, the core issues a programming command to the user interface. Programming command is indicated by a single write command with HSEL="10" and address equal to "08008030"(hex). User can terminate this write command as in normal write cases on the AHB bus. No error signaling is allowed.

During the time the core is waiting for write completion on the AHB bus, the core holds all 4 DT lines to zero to indicate to the host that programming is in progress.

Programming command automatically generated by the core for SD memory and SDIO write. It is not generated for CSA space write.

A.14 CSA Command

CSA read or write commands are issued on the AHB bus as regular read and write command with specific address. There is additional timing requirement for CSA read command. Like all AHB bus requests, the bus master first asserts HBUSREQ

signal. It then waits for HGRANT before issuing the request on AHB bus. From the time HBUSREQ is issued for CSA command, the core must receive bus grant and receives the CSA data within 40 clock cycles.

A.15 Abort Command

The EP560 can abort a read or write command on the AHB bus by issuing early burst termination. Early burst termination is a standard AHB bus protocol and is described in previous section in this appendix. The user logic must terminate the transfer on early burst termination within 10 clock cycles in order to conform to SD timing requirements.

Appendix B. MMC Option

The EP560 SD slave controller from Eureka Technology can be configured to support SD 2.0 standard and/or MultiMedia Card (MMC) 4.2 standard. MMC is very similar to SD/SDIO standard so Eureka's EP560 SD slave controller can be delivered in any of the following 3 configurations:

1. SD memory only support.
2. MMC only support.
3. SDIO only support.
4. SD memory and SDIO (Combo card) support.
5. SD memory and MMC support, selected by hardware mode pin.
5. SD memory, SDIO and MMC support, selected by hardware mode pin.

This document describes the additional features of MMC support for the EP560 IP core. For the basic features of that IP core that is common to both standards, please refer to the EP560 data sheet.

B.1 Signal Descriptions

The major differences between SD2.0 and MMC4.2 is that MMC supports 8-bit data transfer and stream mode data transfer (CMD11 and CMD20). The user interface of the EP560 is versatile enough that it can support both standards without change. The majority of the I/O pins of the EP560 remains unchanged with MMC. The following table lists the few signals that are affected by MMC4.2.

Table 25: Signals for MMC Support

Name	Type	Descriptions
DT[7:0]	I/O	Data line. When MMC support is added, the data line changes from 4 to 8 bit to support 8 bit data transfer.
ECSD_A[7:0], ECSD_B[31:0], ECSD_C[7:0] --- ECSD_Q[7:0]	I	Extended CSD read only data, static input. Extended CSD is a register specific to MMC mode. The register contains mostly read only data and the read only values are provided to the core through this static input. User can hardware these values according to the characteristics of the card.
ECSDR	O	POWER_CLASS from Extended CSD

Table 25: Signals for MMC Support

Name	Type	Descriptions
ECSDS	O	HS_TIMING from Extended CSD
RD_THD[2:0]	I	Read Threshold for stream data read, static input. In steam read mode, in order to prevent data underflow, the user may want to buffer enough data inside the EP560 before the first data is returned to the SD card interface. The encoding of the RD_THD is the following: 000: Buffer 16 words before sending data to host. 001: Buffer 32 words before sending data to host. 010: Buffer 64 words before sending data to host. 011: Buffer 128 words before sending data to host. 111: No Buffer, send data to host as soon as first word arrive from user interface. Others: Reserved
SEL_MMC	I	Select MMC, static input. This signal is present if the core supports both SD mode and MMC mode. This signal is not needed if the core supports only SD or MMC but not both. This signal is “1” to indicate MMC mode and “0” to indicate SD mode. This signal must remain unchanged after reset.

If the EP560 is setup for MMC only with no SD support, the following static inputs related to SD only registers are removed.

Table 26: Signals Not Needed for MMC Only Support

Name	Type	Descriptions
CCCR[152:0]	I	Static input. Lower 153 bits of Card Common Control Registers read only values for SDIO

Table 26: Signals Not Needed for MMC Only Support

Name	Type	Descriptions
CCCR_OUT [153:17]	O	Output from CCCR register, reflect the values of the corresponding bits in the CCCR register. Only the following bits which are writable by the host have valid information. All other bits are hardwired to “0”. The valid bits are: 153, 152, 145, 143:128, 107:104, 97, 69, 63, 61, 57:56, 51:48, 39:32, 23:17.
CCCR_UPPER[127:0]	I	Static input. Upper most 128 bits of Card Common Control Registers read only values for SDIO
EPS1	O	Output from the FBR1 register reflecting the value of the EPS bit (bit 17)
FBR1[143:0]	I	Static input. Function Basic Register read only values for SDIO Function 1.
OCR_SDIO[23:0]	I	Static input. Operation Condition Register read only values, for SDIO.
SCR[63:0]	I	SD Configuration Register read only values, for SD memory.
SDIO_CRD_RDY	I	SDIO card initialization. This signal is high to indication initialization complete and the card is ready.
SRESETI#	O	SDIO soft reset output

B.2 Functional Descriptions

B.2.1 8-bit Data Transfer

The biggest advantage of using MMC is 8-bit data transfer which doubles the data transfer of SD card. When the MMC feature of the EP560 is used, the core automatically supports 8-bit data transfer. The core has 8 bits on the card’s DT line instead of the usual 4. Under software control by the host, the card can be switch between 1, 4 and 8-bit data transfer.

To the user interface and user logic, 8-bit data transfer is completely transparent. Since all the data from the card interface is serially shifted in to the card and transferred to the user logic through a 32-bit data bus, the user interface protocol remains unchanged regardless of the data width on the card side interface. The

only observable differences by the user logic is that ADS# can be asserted earlier between blocks of data because each block takes less time to process.

B.2.2 Bus Testing Procedure

The MMC bus uses the CMD14 and CMD19 commands to detect the data bus width of the device. The test procedure is initiated by the MMC host. The host sends CMD19 followed by a specific pattern on the MMC DT line. The host then sends CMD14 to obtain a response from the MMC slave controller. The test procedure is built-in to the MMC slave controller and no user interaction is required. The MMC slave controller ensures that the highest data bandwidth is maintained on the MMC bus with the widest data path allowed by the system.

B.2.3 Stream Mode Data Transfer

Stream mode data transfer is a unique feature of the MMC bus. In stream data mode transfers data continuously across block boundary. The MMC host issues CMD11, CMD12 and CMD20 commands for stream read and stream write data transfer. The MMC slave controller handles stream data mode automatically. In stream mode operation, data is still read or written from the user logic through the normal ADS#/RDY# bus protocol. In other words, stream mode is transparent to the user logic.

Because data bandwidth on the MMC bus stream mode and user logic may be different, the MMC slave controller uses the internal data buffer as a FIFO to maintain steady data flow.

On write operation, the MMC slave stores the write data from the host in the data buffer and then write to the user logic. The amount of write data transferred per transaction depends on two factors: the capacity of the memory card and whether partial blocks for write are allowed in the CSD (bit 21).

For a card with capacity less than or equal to 2GB and partial blocks allowed, the size of the block sent for transfer varies. Data is written up to the last byte shifted in on the MMC DT bus at the time of abort. The slave controller starts to transfer write data to the user logic when 4 words of data has been accumulated in the data buffer. It uses burst writes of 4 words. If the data transfer rate on the user interface is slow, data starts to accumulate on the data buffer. If 16 or more words has accumulated in the data buffer, the MMC slave controller uses burst of 16 words to write data out to the user logic. When the host issues an abort, the remaining data in the data buffer is transferred to the user logic. If there are more than 16 words in the buffer, burst writes of 16 words are issued to the user logic until there are less than 16 words in the data buffer. Then, if there are more than 4 words in the data buffer, burst writes of 4 words are issued until there are less than 4 words in the data buffer. If there is at least one word in the data buffer, the remaining are issued

in one burst write.

For a card with capacity greater than 2GB and partial blocks allowed, burst writes are issued with 512 bytes per burst, which may be 128 or 129 words, depending on whether the data is aligned to the 32-bit word boundary. 512 bytes is the minimum unit of data for a card with capacity greater than 2GB.

If partial blocks are not allowed, regardless of the memory card capacity, the length of the data sent per burst write is defined by the CSD bits 25 to 22. The number of words sent per burst is the defined length (size in byte) divided by 4 (plus 1 if the data is not aligned to the 32-bit word boundary).

It is important to note that buffer overflow may occur if user logic cannot keep up with the MMC bandwidth. To avoid this, the user logic must be capable of accept, on the average, at same or higher rate at which data is transferred on the MMC bus stream mode (1 bit per clock or 1 word every 32 clocks). If the user logic cannot keep up and buffer overflow occurs, the MMC slave controller sets an error bit, which is read out by the host. The MMC slave controller transfers the data that was in the data buffer before overflow occurred to the user logic.

Buffer underflow is not a concern because the MMC slave controller will only issue write requests to the user logic when data is available to be transferred.

B.2.4 Stream Read Threshold

On read operation, the MMC slave controller issues burst read access of 16 words to read data from the user logic and deposit into its data buffer. Data is then returned to the MMC host at the steady rate of the stream read mode. Buffer overflow is prevented by the MMC slave controller. It does not issue burst read to the user logic if the data buffer is about half full.

It is important to note that buffer underflow may occur if user logic cannot keep up with the MMC bandwidth. To avoid this, the user logic must be capable of accept, on the average, at same or higher rate at which data is transferred on the MMC bus stream mode (1 bit per clock or 1 word every 32 clocks).

Data buffer under-flow may also occur if user logic occasional insert long read latency due to events such as page crossing. To avoid this type of data underflow, it is desirable to accumulate several words of data in the data buffer and then stream data back to the host starts only after the number of data words in the buffer reaches the threshold. The user logic can control the buffer threshold by using the RD_THD static input. In systems with even read latency, a small threshold is preferred. In systems with large read latency when crossing memory physical boundary, a large threshold is preferred.

B.2.5 Programming CID

In MMC, the CID register is one time programmable by the host. Normally this command is reserved for the manufacturer. Because the CID is one time programmable, the user logic is responsible for keeping track of whether or not the CID has been written or not.

To issue the program CID command, ADS# is asserted with CS#="01", address = 08100000 (hex), size = 004(hex), and WR="1". The CID data is on DWR. The user logic responds with RDY# when it has accepted the current word. If the CID has already been written, the user logic asserts ERROR when asserting RDY# and the transfer terminates.

B.2.6 Fast I/O Access

A fast I/O access is a single read or write transfer to the IO address space. The protocol is exactly the same as single read and write as described in the EP560 data sheet. Fast I/O is indicated by CS# signal equals "01" and address bit 31 to 7 all equal to “0”.

B.2.7 MMC Interrupt

The host may put all cards on the MMC bus in interrupt mode by issuing the wait for interrupt command. A card responds back to the host when an internal interrupt trigger event has occurred. The first card to respond with an interrupt response "wins" and all other cards exit interrupt mode.

To issue an interrupt request, ADS# is asserted with CS#="01", address = 08101000 (hex), and WR="1". The user logic responds with RDY# when the internal interrupt trigger event has occured. The interrupt request can be canceled by the ep560, by asserting ABORT#.

B.2.8 MMC Specific Register

The Extended CSD is an MMC specific register. This register is internal to the MMC slave controller. The register contain mostly read only data fields that describes the characteristic of the MMC card. User logic specifies these read only values through the following static inputs. These inputs can be hardwired to the

desirable values by the user or connected to user registers.

Table 27: Extended CSD Static Inputs

Static Input	Extended CSD Field
ECSD_A[7:0]	S_CMD_SET
ECSD_B[31:0]	SEC_COUNT
ECSD_C[7:0]	MIN_PERF_W_8_52
ECSD_D[7:0]	MIN_PERF_R_8_52
ECSD_E[7:0]	MIN_PERF_W_8_26_4_52
ECSD_F[7:0]	MIN_PERF_R_8_26_4_52
ECSD_G[7:0]	MIN_PERF_W_4_26
ECSD_H[7:0]	MIN_PERF_R_4_26
ECSD_I[7:0]	PWR_CL_26_360
ECSD_J[7:0]	PWR_CL_52_360
ECSD_K[7:0]	PWR_CL_26_195
ECSD_L[7:0]	PWR_CL_52_195
ECSD_M[7:0]	CARD_TYPE
ECSD_N[7:0]	CSD_STRUCTURE
ECSD_O[7:0]	EXT_CSD_REV
ECSD_P[7:0]	CMD_SET_REV
ECSD_Q[7:0]	ERASE_MEM_CONT
ECSDR	POWER_CLASS
ECSDS	HS_TIMING

B.3 SD Register Modifications due to MMC

The core function as MMC mode instead of SD memory, the format of the OCR, CID and CSD registers are changed slightly. The following table list the MMC

format with the colored cells indicate changes from SD memory.

Table 28: OCR Registers Format for MMC

Bits	Description	Source
31	Card power up status bit	Input from OCR_MEM
30:29	Access Mode	Input from OCR_MEM
28:24	Reserved	Generated by the controller
23:15	VDD voltage	Input from OCR_MEM
14:8	Reserved	Generated by the controller
7	Low voltage range	Input from OCR_MEM
6:0	Reserved	Generated by the controller

Table 29: CID Register Format for MMC

Bits	Description	Source
127:120	Manufacturer ID	From CID input signal
119:104	OEM/Application ID	From CID input signal
103:56	Product name	From CID input signal
55:48	Product revision	From CID input signal
47:16	Product serial number	From CID input signal
15:8	Manufacturing date	From CID input signal
7:1	CRC7 checksum	From CID input signal
0	Reserved	Generated by the controller

Table 30: CSD Register Format for MMC

Bits	Description	Source
127:126	CSD structure To indicate version 1.0, these two bits should be “00”	From CSD input signal
125:122	SPEC_VERS	From CSD input signal
121:120	Reserved	Generated by the controller
119:112	TAAC	From CSD input signal
111:104	NSAC	From CSD input signal
103:96	TRAN_SPEED	From CSD input signal
95:84	CCC	From CSD input signal
83:80	READ_BL_LEN	From CSD input signal
79	READ_BL_PARTIAL	From CSD input signal
78	WRITE_BLK_MISALIGN	From CSD input signal
77	READ_BLK_MISALIGN	From CSD input signal
76	DSR_IMP	From CSD input signal
75:74	Reserved	Generated by the controller
73:62	C_SIZE	From CSD input signal
61:59	VDD_R_CURR_MIN	From CSD input signal
58:56	VDD_R_CURR_MAX	From CSD input signal
55:53	VDD_W_CURR_MIN	From CSD input signal
52:50	VDD_W_CURR_MAX	From CSD input signal
49:47	C_SIZE_MULT	From CSD input signal
46:42	ERASE_GRP_SIZE	From CSD input signal
41:37	ERASE_GRP_MULT	From CSD input signal
36:32	WP_GRP_SIZE	From CSD input signal

Table 30: CSD Register Format for MMC

Bits	Description	Source
31	WP_GRP_ENABLE	From CSD input signal
30:29	DEFAULT_ECC	From CSD input signal
28:26	R2W_FACTOR	From CSD input signal
25:22	WRITE_BL_LEN	From CSD input signal
21	WRITE_BL_PARTIAL	From CSD input signal
20:17	Reserved	Generated by the controller
16	CONTENT_PROT_APP	From CSD input signal
15	FILE_FORMAT_GRP	From CSD input signal
14	COPY	From CSD input signal
13	PERM_WRITE_PROTECT	From CSD input signal
12	TMP_WRITE_PROTECT	Generated by the controller
11:10	FILE_FORMAT	From CSD input signal
9:8	ECC	From CSD input signal
7:1	CRC	From CSD input signal
0	Reserved	Generated by the controller

EP560 SDIO/SD memory Slave Controller

Data Sheet

Eureka Technology Inc.
4962 El Camino Real
Los Altos, CA 94022, USA

Tel: 1 650 960 3800
Fax: 1 650 960 3805
email: info@eurekatech.com
http://www.eurekatech.com